Database design: ER diagrams and data dictionary

Dubai Real Estate CRM · Document 03 · Version 1.2 · 11 October 2026 · Matches SRS v1.2 · PostgreSQL 16

Generated directly from the tested database, so the diagrams and the dictionary always match the real schema.

77 tables	925 columns
230 relationships	51/51 rule tests passed
10 sections	

How the database is designed

In simple words: every customer's data is locked inside its own "tenant", so one brokerage can never see another's. Important rules (no double booking, no double import, reasons for Lost, locked invoices) are enforced by the database itself, so even a bug in the app cannot break them. History tables can only be added to, never edited.

Design rules (for developers)

- Every business table has `tenant_id` and `UNIQUE (tenant_id, id)`; child tables use composite foreign keys `(tenant_id, x_id)`, so cross-tenant links are impossible.
- Row-level security on all tenant tables: the API sets `app.tenant_id` per transaction.
- Primary keys: `uuid` (app generates UUIDv7). Times: `timestamptz` (UTC). Money: `numeric(14,2)` + currency.
- Value lists are `text + CHECK`, easy to extend with a migration. Configurable lists (stages, reasons, roles) are tables.
- Each module owns its tables; other modules use its service, never its tables.

Changes in v1.2 and v1.1

- **v1.2:** new Subscriptions & billing section (11 tables): `plans`, `subscriptions`, `subscription_events`, `billing_invoices`, `billing_invoice_lines`, `billing_payments`, `tenant_usage_daily`, `platform_admins`, `platform_settings`, `sales_enquiries`, `platform_audit_logs`. `tenants`: RERA BRN, licence file and verification added, `plan` moved to `subscriptions`. Seat limit enforced by the database. New role `crm_platform`.
- New `lead_shares`: view/edit sharing.
- New `booking_pages`: replaces Calendly.
- Removed `lead_duplicates` and merge fields; repeat enquiries go to the existing lead (`is_re_enquiry`, `enquiry_count`).
- 10-minute rule: `leads.response_due_at`; moves logged as `no_response` / `admin_fallback`.
- `users.receives_leads`: Admin on/off switch.
- Auto-close fields removed. Zoom, Calendly, respond.io, Make removed; Dubizzle added.
- Exports: `leads.export`, `reports.export`, `attendance.export`, Admin only by default.

Database files

- `schema.sql`: tables, relationships, rules, indexes
- `security.sql`: roles, row-level security, append-only history, invoice locks
- `seed.sql`: currencies, permissions, default roles, stages and reasons for each new tenant

Database roles

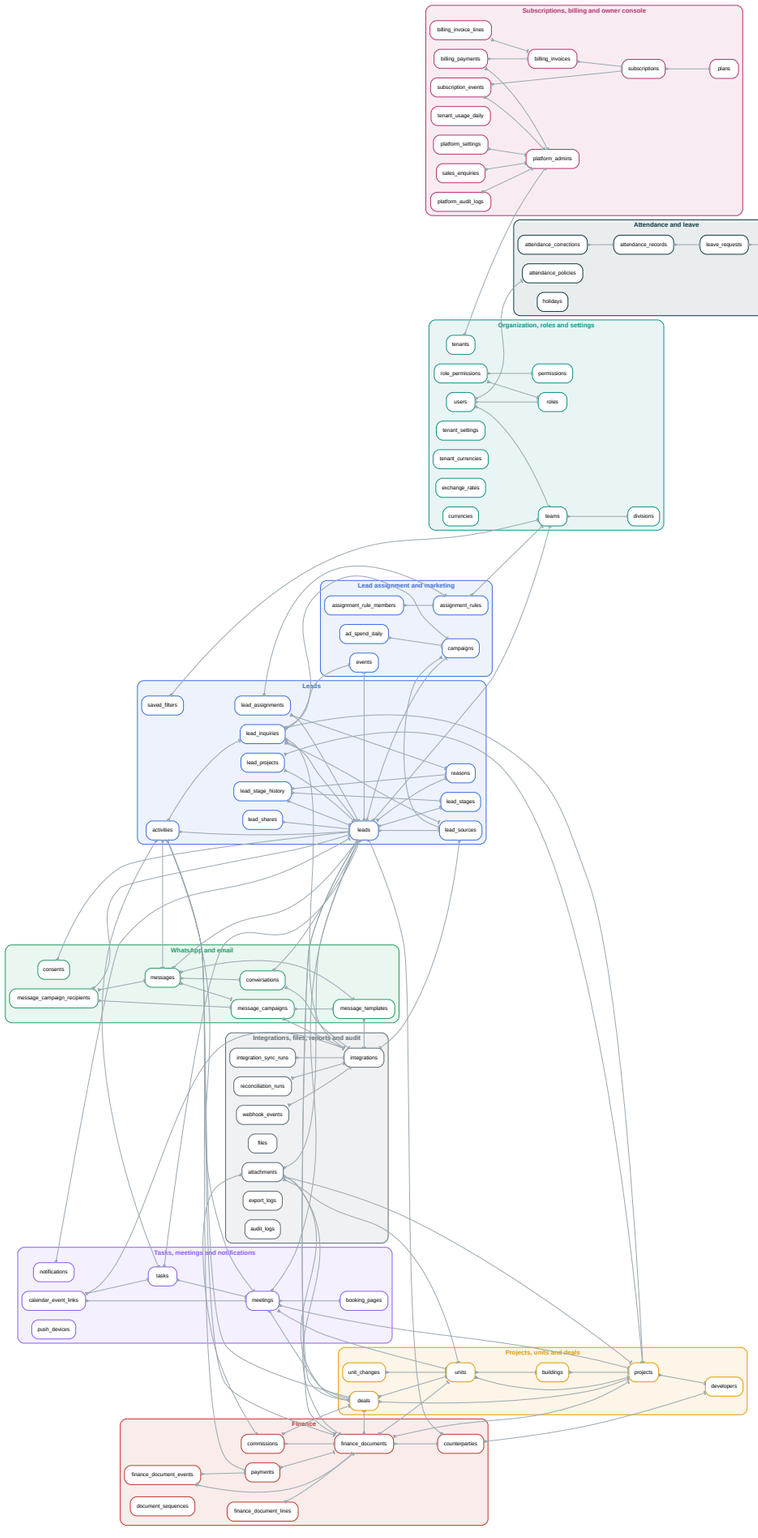
- `crm_app`: API, one tenant per transaction
- `crm_worker`: background jobs across tenants
- `crm_readonly`: support and reporting
- `crm_platform`: owner console and Stripe webhooks

Rule tests run on the real database

RULE	RESULT
Tenant A user sees only tenant A leads	✓ PASS
No tenant set: sees nothing	✓ PASS
Tenant A cannot insert a row for tenant B	✓ PASS
Tenant A cannot read tenant B's lead by ID	✓ PASS
Lead cannot point to another tenant's user	✓ PASS
Same submission cannot be imported twice	✓ PASS
Stage category must match the stage	✓ PASS
Lost requires a reason	✓ PASS
Lost with a reason is accepted	✓ PASS
Open assigned lead must have a next follow-up	✓ PASS
Response deadline only when the lead has an owner	✓ PASS
10-minute rule move is logged (no_response)	✓ PASS
Admin fallback is logged	✓ PASS
Manual reassignment needs a reason	✓ PASS
Lead can be shared as 'edit'	✓ PASS
Share access must be view or edit	✓ PASS
Agent cannot be double-booked	✓ PASS
Booking-page meeting must reference a booking page	✓ PASS
Zoom is no longer an allowed channel	✓ PASS
Deal unit must belong to the deal's project	✓ PASS
A unit cannot be booked by two deals	✓ PASS
Invoice total and balance are calculated	✓ PASS
Draft invoice cannot be 'paid' without approval	✓ PASS
Approved invoice amounts are frozen	✓ PASS

RULE	RESULT
Lines of an approved invoice cannot change	✓ PASS
Purchase order cannot be 'paid'	✓ PASS
History is append-only (audit log)	✓ PASS
Enquiries cannot be edited	✓ PASS
Overlapping approved leave is blocked	✓ PASS
Nobody approves their own leave	✓ PASS
A file attaches to exactly one record	✓ PASS
Only Admin has export permissions (default)	✓ PASS
Admin can switch off lead assignment for a user	✓ PASS
Active users cannot exceed paid seats	✓ PASS
Invited (not active) users don't use a seat	✓ PASS
Seats cannot drop below active users	✓ PASS
Admin can add seats	✓ PASS
Tenant cannot change its own billing status	✓ PASS
Tenant cannot mark itself verified	✓ PASS
Tenant cannot create its own invoices or payments	✓ PASS
Tenant sees only its own subscription invoices	✓ PASS
Trial needs a card on file	✓ PASS
Payment failure needs a grace end date	✓ PASS
Cancelled subscription has a data deletion date	✓ PASS
Bank transfer counts only when confirmed by the platform team	✓ PASS
Invoice VAT total is calculated (AED 396 + 5%)	✓ PASS
Issued subscription invoice cannot be edited	✓ PASS
Brokerage needs trade licence and RERA ORN	✓ PASS
Individual agent needs RERA broker card (BRN)	✓ PASS
Subscription history is append-only	✓ PASS
App cannot see platform admins	✓ PASS

Overview: all tables by section



Lines show relationships between tables. Links to tenants, users, files and currencies are left out here to keep it readable; they are in each section's diagram.

Contents

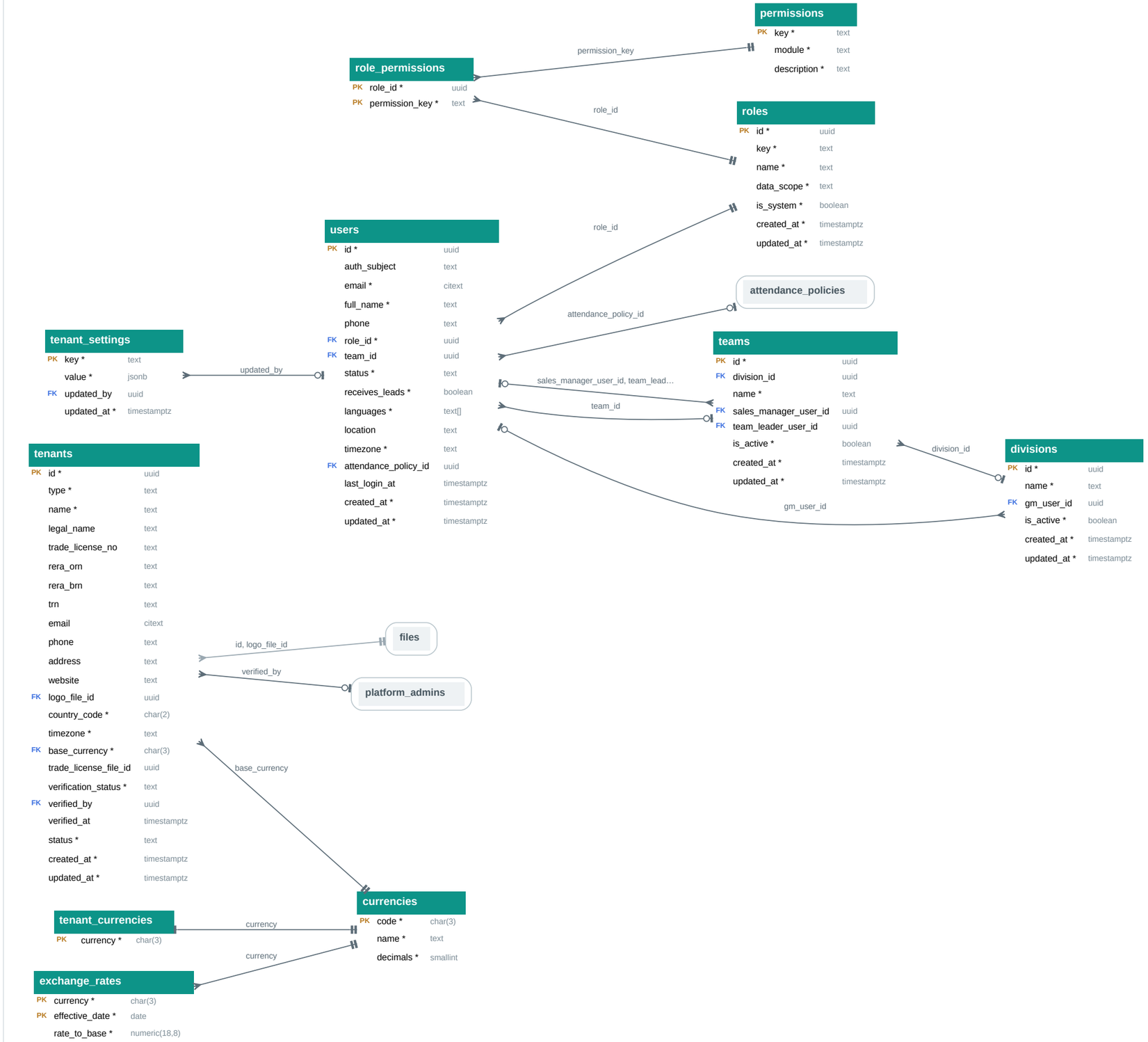
1. Organization, roles and settings (11 tables)
2. Leads (11 tables)
3. Lead assignment and marketing (5 tables)
4. Tasks, meetings and notifications (6 tables)
5. WhatsApp and email (6 tables)
6. Projects, units and deals (6 tables)
7. Finance (7 tables)
8. Attendance and leave (6 tables)
9. Subscriptions, billing and owner console (11 tables)
10. Integrations, files, reports and audit (8 tables)

How to read the ER diagrams: each box is a table. **PK** primary key · **FK** link to another table · * required. Line ends: crow's foot = many, single bar = one, circle = optional.

1 Organization, roles and settings

In simple words: Each customer company (tenant) has its own users, teams and roles. Roles decide what each person can do; Admin can create new roles such as HR.

Tables: tenants · roles · role_permissions · permissions · users · divisions · teams · tenant_settings · tenant_currencies · exchange_rates · currencies



ER diagram: Organization, roles and settings. Grey boxes are tables from other sections. tenant_id is hidden (every table belongs to a tenant).

tenants

One customer account: a brokerage (B2B) or an individual agent (B2C).

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK FK → files	auto (uuid)	Tenant ID
type	text	Yes			B2B brokerage or B2C individual agent
name	text	Yes			Display name
legal_name	text				Registered company name
trade_license_no	text				Dubai trade licence number
rera_orn	text				RERA office registration number (ORN), required for brokerages
rera_brn	text				RERA broker card number (BRN), required for individual agents
trn	text				UAE VAT tax registration number
email	citext				Company email
phone	text				Company phone (E.164)
address	text				Company address
website	text				Company website
logo_file_id	uuid		FK → files		Logo image (files)
country_code	char(2)	Yes		'AE'	ISO country code
timezone	text	Yes		'Asia/Dubai'	Default IANA time zone
base_currency	char(3)	Yes	FK → currencies	'AED'	Reporting currency
trade_license_file_id	uuid				Uploaded trade licence or broker card (files)
verification_status	text	Yes		'pending'	RERA / licence check by the platform team
verified_by	uuid		FK → platform_admins		Platform admin who verified
verified_at	timestamptz				Verification time
status	text	Yes		'active'	Platform switch (suspended by owner; closed after cancellation); billing state is in subscriptions
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Check:** `((type = ANY (['brokerage', 'individual'])))`
- **Check:** `(((verification_status = 'verified') = (verified_at IS NOT NULL)))`
- **Check:** `(((type <> 'individual') OR (rera_brn IS NOT NULL)))`
- **Check:** `(((type <> 'brokerage') OR ((trade_license_no IS NOT NULL) AND (rera_orn IS NOT NULL))))`
- **Check:** `((status = ANY (['active', 'suspended', 'closed'])))`
- **Check:** `((verification_status = ANY (['pending', 'verified', 'rejected'])))`
- **Check:** `((country_code ~ '^[A-Z]{2}$'))`

roles

Roles per tenant (Director, GM, Sales Manager, Team Leader, Agent, Admin, Finance) and their data scope.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Role ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
key	text	Yes			Stable key, e.g. sales_agent
name	text	Yes			Display name
data_scope	text	Yes			Which records the role can see
is_system	boolean	Yes		false	Built-in role (cannot be deleted)
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** `(tenant_id, id)`
- **Unique:** `(tenant_id, key)`
- **Check:** `((data_scope = ANY (['own', 'team', 'division', 'company'])))`
- **Check:** `((key ~ '^[a-z_]+$'))`

role_permissions

Which actions each role may perform.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
tenant_id	uuid	Yes			Owning tenant
role_id	uuid	Yes	PK FK → roles		Role
permission_key	text	Yes	PK FK → permissions		Granted permission

- **On delete:** `role_id → roles: cascade`

permissions

Catalogue of actions that can be granted to roles.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
key	text	Yes	PK		Permission key, e.g. leads.export
module	text	Yes			Module the action belongs to
description	text	Yes			What the permission allows

- **Check:** `((key ~ '^[a-z_]+\.[a-z_]+$'))`

users

People who log in. Each user belongs to one tenant and has one role.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	User ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
auth_subject	text				Login provider user ID (Cognito sub); empty until invite accepted
email	citext	Yes			Login email
full_name	text	Yes			Full name
phone	text				Mobile (E.164)

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
role_id	uuid	Yes	FK → roles		Role
team_id	uuid		FK → teams		Team (agents and team leaders)
status	text	Yes		'invited'	Account status
receives_leads	boolean	Yes		true	Admin switch: user gets new leads (existing leads unaffected)
languages	text[]	Yes		'{}'	Spoken languages (ISO 639-1) for assignment rules
location	text				Office or area, for assignment rules
timezone	text	Yes		'Asia/Dubai'	Display time zone
attendance_policy_id	uuid		FK → attendance_policies		Attendance policy override (default policy if empty)
last_login_at	timestamptz				Last successful login
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** (auth_subject)
- **Unique:** (tenant_id, email)
- **Unique:** (tenant_id, id)
- **Check:** ((phone ~ '^\+[1-9][0-9]{6,14}$'))
- **Check:** ((status = ANY (['invited', 'active', 'inactive'])))
- **Index:** (tenant_id, role_id)
- **Index:** (tenant_id, team_id)

divisions

Business divisions, each run by one General Manager.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Division ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
name	text	Yes			Division name
gm_user_id	uuid		FK → users		General Manager
is_active	boolean	Yes		true	Active flag
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** (tenant_id, id)
- **Unique:** (tenant_id, name)

teams

Sales teams. A Sales Manager can manage several teams; each team has one Team Leader.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Team ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
division_id	uuid		FK → divisions		Division the team belongs to
name	text	Yes			Team name
sales_manager_user_id	uuid		FK → users		Sales Manager responsible for the team
team_leader_user_id	uuid		FK → users		Team Leader
is_active	boolean	Yes		true	Active flag
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** (tenant_id, name)
- **Unique:** (tenant_id, id)
- **Index:** (tenant_id, division_id)
- **Index:** (tenant_id, sales_manager_user_id)

tenant_settings

Tenant configuration values (response time, auto-close policy, scoring weights, work hours, etc.).

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
tenant_id	uuid	Yes	PK FK → tenants		Owning tenant
key	text	Yes	PK		Setting key, e.g. leads.auto_close
value	jsonb	Yes			Setting value (validated by the application)
updated_by	uuid		FK → users		Last changed by
updated_at	timestamptz	Yes		now()	Last update time

- **Check:** ((key ~ '^[a-z_.]+$'))

tenant_currencies

Currencies enabled for a tenant.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
tenant_id	uuid	Yes	PK FK → tenants		Owning tenant
currency	char(3)	Yes	PK FK → currencies		Enabled currency

exchange_rates

Exchange rates to the tenant base currency, used for report totals.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
tenant_id	uuid	Yes	PK FK → tenants		Owning tenant
currency	char(3)	Yes	PK FK → currencies		Currency
effective_date	date	Yes	PK		Rate valid from this date
rate_to_base	numeric(18,8)	Yes			1 unit of currency = rate_to_base units of base currency

- **Check:** ((rate_to_base > (0)))

currencies

ISO 4217 currencies the system can use.

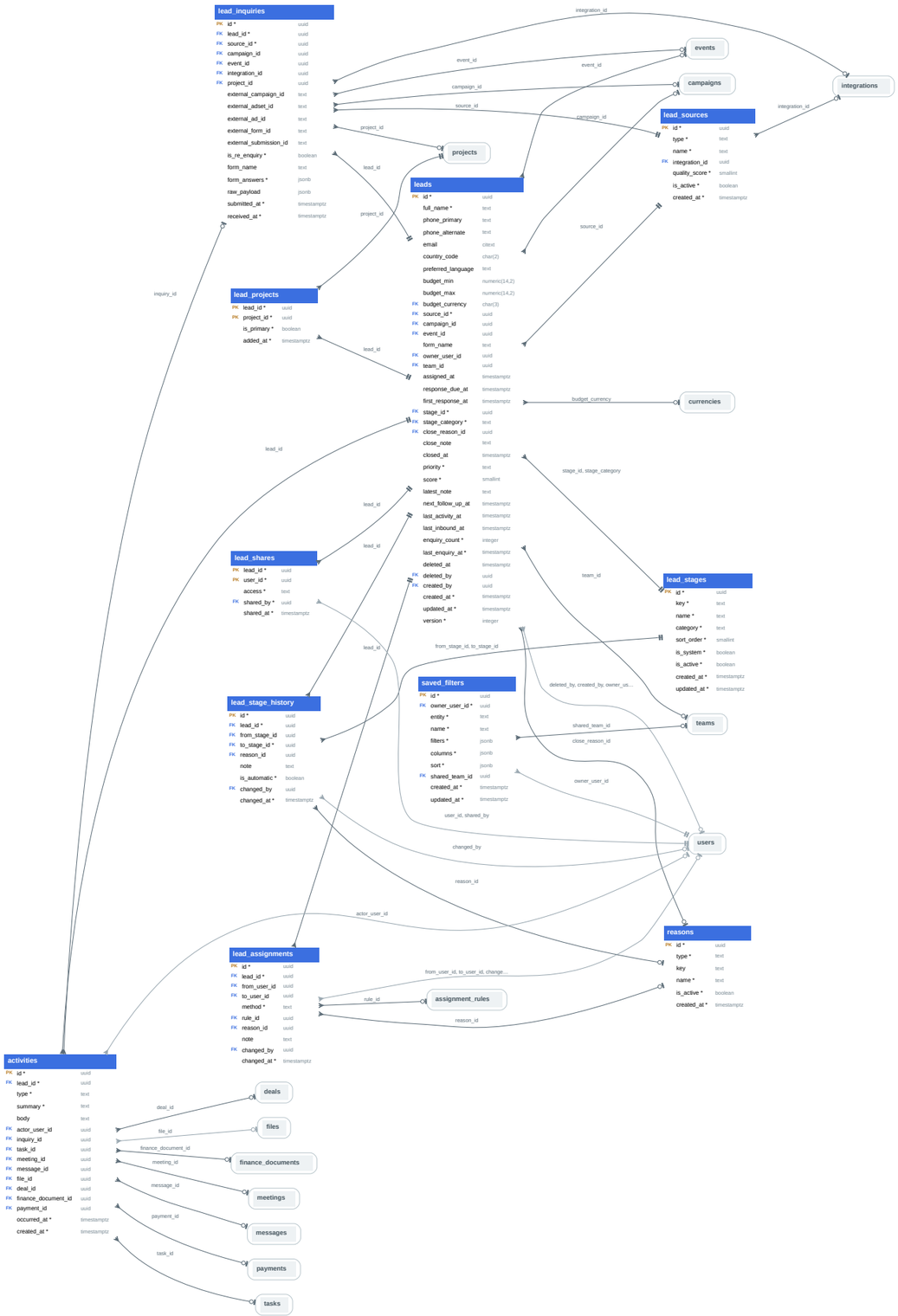
COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
code	char(3)	Yes	PK		ISO currency code, e.g. AED
name	text	Yes			Currency name
decimals	smallint	Yes		2	Decimal places

- **Check:** `(((decimals >= 0) AND (decimals <= 3)))`
- **Check:** `((code ~ '^[A-Z]{3}$'))`

2 Leads

In simple words: Every buyer is one lead. Each time they enquire, an enquiry row is added (repeat enquiries go to the same lead and agent). Every stage change, owner change and share is recorded and can never be edited.

Tables: leads · lead_inquiries · lead_projects · lead_stage_history · lead_assignments · lead_shares · activities · saved_filters · lead_stages · reasons · lead_sources



ER diagram: Leads. Grey boxes are tables from other sections. tenant_id is hidden (every table belongs to a tenant).

leads

One person interested in property. stage_category is kept in step with the stage by a foreign key.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Lead ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
full_name	text	Yes			Full name
phone_primary	text				Primary phone (E.164)
phone_alternate	text				Alternate phone (E.164)
email	citext				Email (case-insensitive)
country_code	char(2)				Country of residence (ISO)
preferred_language	text				Preferred language (ISO 639-1)
budget_min	numeric(14,2)				Budget from
budget_max	numeric(14,2)				Budget to
budget_currency	char(3)		FK → currencies		Budget currency
source_id	uuid	Yes	FK → lead_sources		Original source
campaign_id	uuid		FK → campaigns		Original campaign
event_id	uuid		FK → events		Event or roadshow
form_name	text				Form name of the latest inquiry
owner_user_id	uuid		FK → users		Responsible agent (empty = Unassigned queue)
team_id	uuid		FK → teams		Owner's team at assignment (for team scope)
assigned_at	timestamptz				When the current owner was assigned
response_due_at	timestamptz				Deadline for the owner's first action (10-minute rule); empty once acted
first_response_at	timestamptz				Owner's first logged action after assignment
stage_id	uuid	Yes	FK → lead_stages		Current stage
stage_category	text	Yes	FK → lead_stages		Category of the current stage (kept in step by FK)
close_reason_id	uuid		FK → reasons		Reason for Lost / Invalid
close_note	text				Extra detail for Lost / Invalid
closed_at	timestamptz				When the lead reached a terminal stage
priority	text	Yes		'medium'	Agent-set priority
score	smallint	Yes		0	Lead score
latest_note	text				Copy of the latest note (for the list view)
next_follow_up_at	timestamptz				Next planned action
last_activity_at	timestamptz				Last logged activity of any kind
last_inbound_at	timestamptz				Last reply / inbound contact from the lead
enquiry_count	integer	Yes		1	Number of enquiries (more than 1 = re-enquired)
last_enquiry_at	timestamptz	Yes		now()	Time of the latest enquiry

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
deleted_at	timestamptz				Soft delete time
deleted_by	uuid		FK → users		Deleted by
created_by	uuid		FK → users		Created by (empty = integration)
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time
version	integer	Yes		1	Optimistic lock counter

- **Unique:** (tenant_id, id)
- **Check:** (((deleted_by IS NULL) OR (deleted_at IS NOT NULL)))
- **Check:** (((phone_primary IS NOT NULL) OR (email IS NOT NULL)))
- **Check:** (((response_due_at IS NULL) OR (owner_user_id IS NOT NULL)))
- **Check:** (((owner_user_id IS NULL) = (assigned_at IS NULL)))
- **Check:** (((stage_category <> 'open') OR (owner_user_id IS NULL) OR (next_follow_up_at IS NOT NULL) OR (deleted_at IS NOT NULL)))
- **Check:** (((stage_category = 'open') = (closed_at IS NULL)))
- **Check:** (((stage_category <> ALL (['lost', 'invalid'])) OR (close_reason_id IS NOT NULL)))
- **Check:** ((((budget_min IS NULL) AND (budget_max IS NULL)) OR (budget_currency IS NOT NULL)))
- **Check:** (((budget_min IS NULL) OR (budget_max IS NULL) OR (budget_max >= budget_min)))
- **Check:** ((enquiry_count >= 1))
- **Check:** (((score >= 0) AND (score <= 100)))
- **Check:** ((priority = ANY (['low', 'medium', 'high'])))
- **Check:** ((stage_category = ANY (['open', 'won', 'lost', 'invalid'])))
- **Check:** ((budget_max >= (0)))
- **Check:** ((country_code ~ '^[A-Z]{2}$'))
- **Check:** ((phone_alternate ~ '^\+[1-9][0-9]{6,14}$'))
- **Check:** ((phone_primary ~ '^\+[1-9][0-9]{6,14}$'))
- **Check:** ((budget_min >= (0)))
- **Index:** (tenant_id, campaign_id, created_at)
- **Index:** (tenant_id, created_at DESC)
- **Index:** (tenant_id, email) WHERE (deleted_at IS NULL)
- **Index:** (tenant_id, event_id) WHERE (event_id IS NOT NULL)
- **Index:** (tenant_id, next_follow_up_at) WHERE ((deleted_at IS NULL) AND (stage_category = 'open'::text))
- **Index:** (full_name gin_trgm_ops)
- **Index:** (tenant_id, owner_user_id, next_follow_up_at) WHERE (deleted_at IS NULL)
- **Index:** (tenant_id, phone_alternate) WHERE ((deleted_at IS NULL) AND (phone_alternate IS NOT NULL))
- **Index:** (tenant_id, phone_primary) WHERE (deleted_at IS NULL)
- **Index:** (tenant_id, last_enquiry_at DESC) WHERE (enquiry_count > 1)
- **Index:** (response_due_at) WHERE (response_due_at IS NOT NULL)
- **Index:** (tenant_id, score DESC, next_follow_up_at) WHERE (deleted_at IS NULL)
- **Index:** (tenant_id, source_id, created_at)
- **Index:** (tenant_id, stage_id) WHERE (deleted_at IS NULL)
- **Index:** (tenant_id, team_id) WHERE (deleted_at IS NULL)
- **Index:** (tenant_id, created_at) WHERE ((owner_user_id IS NULL) AND (stage_category = 'open'::text) AND (deleted_at IS NULL))

lead_inquiries

Every enquiry received for a lead, with original source IDs and form answers. Never changed after saving.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Inquiry ID
tenant_id	uuid	Yes			Owning tenant
lead_id	uuid	Yes	FK → leads		Lead (repeat enquiries are added to the existing lead)
source_id	uuid	Yes	FK → lead_sources		Source
campaign_id	uuid		FK → campaigns		Campaign
event_id	uuid		FK → events		Event or roadshow
integration_id	uuid		FK → integrations		Integration that delivered it
project_id	uuid		FK → projects		Project asked about
external_campaign_id	text				Platform campaign ID
external_adset_id	text				Platform ad set ID
external_ad_id	text				Platform ad ID
external_form_id	text				Platform form ID
external_submission_id	text				Platform submission / lead ID (prevents double import)
is_re_enquiry	boolean	Yes		false	Matched an existing lead (Re-enquired)
form_name	text				Form name
form_answers	jsonb	Yes		'{}'	Original answers exactly as received
raw_payload	jsonb				Full original payload
submitted_at	timestamptz	Yes			Original submission time at the source
received_at	timestamptz	Yes		now()	Time the CRM received it

- **Unique:** `(tenant_id, id)`
- **Index:** `(tenant_id, campaign_id, submitted_at)`
- **Index:** `(tenant_id, lead_id, submitted_at DESC)`
- **Index:** `Unique (tenant_id, source_id, external_submission_id) WHERE (external_submission_id IS NOT NULL)`

lead_projects

Projects a lead is interested in. One can be marked primary (shown in the list).

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
tenant_id	uuid	Yes			Owning tenant
lead_id	uuid	Yes	PK FK → leads		Lead
project_id	uuid	Yes	PK FK → projects		Project
is_primary	boolean	Yes		false	Main project of interest
added_at	timestamptz	Yes		now()	Linked time

- **Index:** `Unique (lead_id) WHERE is_primary`
- **Index:** `(tenant_id, project_id)`
- **On delete:** `lead_id → leads: cascade`

lead_stage_history

Every stage change (used for the funnel and audit). Never changed after saving.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	History ID
tenant_id	uuid	Yes			Owning tenant
lead_id	uuid	Yes	FK → leads		Lead
from_stage_id	uuid		FK → lead_stages		Previous stage (empty for the first stage)
to_stage_id	uuid	Yes	FK → lead_stages		New stage
reason_id	uuid		FK → reasons		Reason (required for Lost / Invalid)
note	text				Comment
is_automatic	boolean	Yes		false	Changed by automation (auto-close, reopen, booking)
changed_by	uuid		FK → users		Changed by (empty = system)
changed_at	timestamptz	Yes		now()	Change time

- **Index:** (tenant_id, lead_id, changed_at)
- **Index:** (tenant_id, to_stage_id, changed_at)

lead_assignments

Every owner change: previous owner, new owner, who, when, why. Never changed after saving.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Assignment ID
tenant_id	uuid	Yes			Owning tenant
lead_id	uuid	Yes	FK → leads		Lead
from_user_id	uuid		FK → users		Previous owner (empty = was unassigned)
to_user_id	uuid		FK → users		New owner (empty = moved to Unassigned)
method	text	Yes			How it was assigned (no_response = 10-minute rule; admin_fallback = whole team missed it)
rule_id	uuid		FK → assignment_rules		Rule used
reason_id	uuid		FK → reasons		Reassignment reason (required for manual reassignment)
note	text				Comment
changed_by	uuid		FK → users		Changed by (empty = system)
changed_at	timestamptz	Yes		now()	Change time

- **Check:** ((from_user_id IS DISTINCT FROM to_user_id))
- **Check:** (((method <> 'manual') OR (from_user_id IS NULL) OR (reason_id IS NOT NULL)))
- **Check:** ((method = ANY (['manual', 'round_robin', 'rule', 'no_response', 'admin_fallback', 'user_deactivated', 're_enquiry'])))
- **Index:** (tenant_id, lead_id, changed_at)

lead_shares

Secondary users who can see (view) or work on (edit) a lead. The owner stays the owner.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
tenant_id	uuid	Yes			Owning tenant
lead_id	uuid	Yes	PK FK → leads		Lead
user_id	uuid	Yes	PK FK → users		Secondary user
access	text	Yes			View only or Can edit
shared_by	uuid	Yes	FK → users		Who shared it (owner or manager)
shared_at	timestamptz	Yes		now()	When shared

- **Check:** ((access = ANY (['view', 'edit'])))
- **Index:** (tenant_id, user_id)
- **On delete:** lead_id → leads: cascade
- **On delete:** user_id → users: cascade

activities

The lead timeline: one row per event (note, call, message, meeting, stage change, invoice…). Typed links point to the source row.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Activity ID
tenant_id	uuid	Yes			Owning tenant
lead_id	uuid	Yes	FK → leads		Lead
type	text	Yes			Event type
summary	text	Yes			One-line text shown on the timeline
body	text				Full text (notes)
actor_user_id	uuid		FK → users		Who did it (empty = system / lead)
inquiry_id	uuid		FK → lead_inquiries		Related inquiry
task_id	uuid		FK → tasks		Related task / call
meeting_id	uuid		FK → meetings		Related meeting
message_id	uuid		FK → messages		Related message
file_id	uuid		FK → files		Related file
deal_id	uuid		FK → deals		Related deal
finance_document_id	uuid		FK → finance_documents		Related invoice / PO
payment_id	uuid		FK → payments		Related payment
occurred_at	timestamptz	Yes		now()	When it happened
created_at	timestamptz	Yes		now()	Recorded time

- **Check:** ((type = ANY (['inquiry', 'note', 'call', 'stage_change', 'owner_change', 'whatsapp', 'email', 'meeting', 'viewing', 'attachment', 'deal', 'eoi', 'booking', 'invoice', 'purchase_order', 'payment', 're_enquiry', 'share', 'system'])))
- **Check:** (((type <> 'note') OR (body IS NOT NULL)))
- **Index:** (tenant_id, lead_id, occurred_at DESC)

saved_filters

Saved list filters and column choices, private or shared with a team.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Filter ID
tenant_id	uuid	Yes			Owning tenant
owner_user_id	uuid	Yes	FK → users		Created by
entity	text	Yes			List it applies to
name	text	Yes			Filter name
filters	jsonb	Yes		'{}'	Filter values
columns	jsonb	Yes		'[]'	Visible columns in order
sort	jsonb	Yes		'[]'	Sort order
shared_team_id	uuid		FK → teams		Team it is shared with (empty = private)
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** (owner_user_id, entity, name)
- **Check:** ((entity = ANY (['leads', 'tasks', 'units', 'finance_documents', 'messages', 'attendance'])))
- **On delete:** owner_user_id → users: cascade

lead_stages

Configurable lead stages. category drives rules: open, won, lost, invalid.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Stage ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
key	text	Yes			Stable key, e.g. no_response
name	text	Yes			Display name
category	text	Yes			Open or terminal category
sort_order	smallint	Yes			Order in the stage bar
is_system	boolean	Yes		false	Built-in stage used by automation (cannot be deleted)
is_active	boolean	Yes		true	Active flag
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** (tenant_id, id)
- **Unique:** (tenant_id, key)
- **Unique:** (tenant_id, id, category)
- **Check:** ((key ~ '^[a-z_]+$'))
- **Check:** ((category = ANY (['open', 'won', 'lost', 'invalid'])))

reasons

Configurable reasons for Lost, Invalid and reassignment.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Reason ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
type	text	Yes			Where the reason is used
key	text				Stable key for system reasons, e.g. no_response
name	text	Yes			Display text
is_active	boolean	Yes		true	Active flag
created_at	timestamptz	Yes		now()	Created time

- **Unique:** (tenant_id, type, name)
- **Unique:** (tenant_id, type, key)
- **Unique:** (tenant_id, id)
- **Check:** ((type = ANY (['lost', 'invalid', 'reassignment'])))
- **Check:** ((key ~ '^[a-z_]+$'))

lead_sources

Where leads come from (Meta, portals, website, import, events...). quality_score feeds lead scoring.

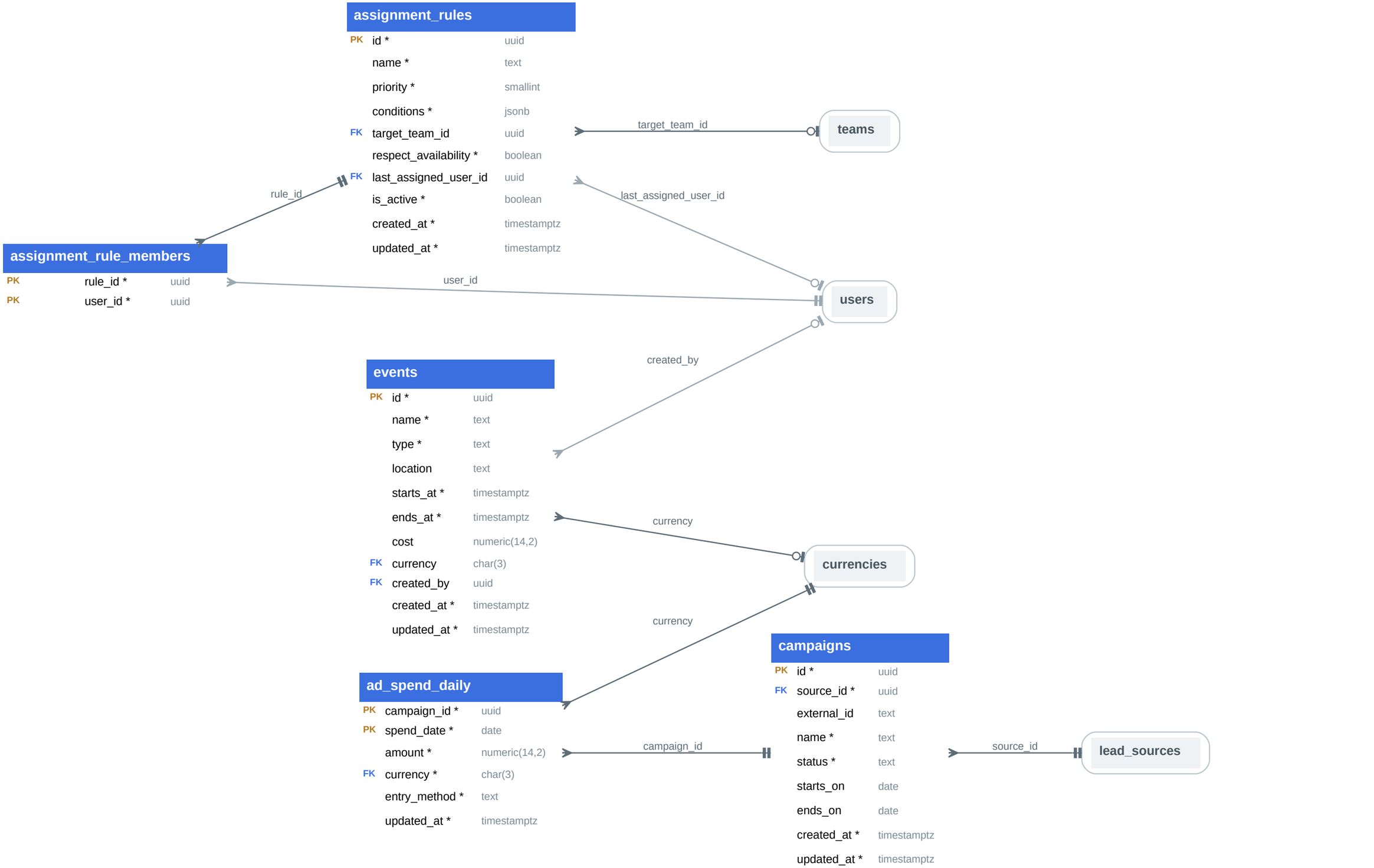
COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Source ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
type	text	Yes			Source type
name	text	Yes			Display name
integration_id	uuid		FK → integrations		Integration that delivers this source
quality_score	smallint	Yes		50	Source quality used in scoring
is_active	boolean	Yes		true	Active flag
created_at	timestamptz	Yes		now()	Created time

- **Unique:** (tenant_id, id)
- **Unique:** (tenant_id, name)
- **Check:** (((quality_score >= 0) AND (quality_score <= 100)))
- **Check:** ((type = ANY (['meta', 'google_ads', 'linkedin', 'snapchat', 'tiktok', 'website', 'webhook', 'property_finder', 'bayut', 'dubizzle', 'whatsapp', 'import', 'manual', 'event', 'referral', 'other'])))

3 Lead assignment and marketing

In simple words: Rules decide which agent gets a new lead. Campaigns, daily ad spend and events tell us where leads came from and what they cost.

Tables: assignment_rules · assignment_rule_members · campaigns · ad_spend_daily · events



ER diagram: Lead assignment and marketing. Grey boxes are tables from other sections. tenant_id is hidden (every table belongs to a tenant).

assignment_rules

Rules checked top to bottom to choose the owner of a new lead.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Rule ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
name	text	Yes			Rule name
priority	smallint	Yes			Order (1 = checked first)
conditions	jsonb	Yes		'{}'	Match conditions: project_ids, campaign_ids, source_ids, languages, locations
target_team_id	uuid		FK → teams		Round-robin across this team (or across rule members)
respect_availability	boolean	Yes		true	Skip unavailable / on-leave agents
last_assigned_user_id	uuid		FK → users		Round-robin pointer
is_active	boolean	Yes		true	Active flag
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** `(tenant_id, id)`
- **Unique:** `(tenant_id, priority) DEFERRABLE INITIALLY DEFERRED`

assignment_rule_members

Specific agents a rule rotates between (used when no target team is set).

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
tenant_id	uuid	Yes			Owning tenant
rule_id	uuid	Yes	PK FK → assignment_rules		Rule
user_id	uuid	Yes	PK FK → users		Agent

- **On delete:** `rule_id → assignment_rules: cascade`

campaigns

Ad campaigns from Meta, Google, etc., or manual campaigns.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Campaign ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
source_id	uuid	Yes	FK → lead_sources		Source / platform
external_id	text				Campaign ID at the platform
name	text	Yes			Campaign name
status	text	Yes		'active'	Status
starts_on	date				Start date
ends_on	date				End date
created_at	timestamptz	Yes		now()	Created time

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** (tenant_id, source_id, external_id)
- **Unique:** (tenant_id, id)
- **Check:** (((ends_on IS NULL) OR (starts_on IS NULL) OR (ends_on >= starts_on)))
- **Check:** ((status = ANY (['active', 'paused', 'ended'])))

ad_spend_daily

Daily ad spend per campaign, imported from ad accounts or entered manually.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
tenant_id	uuid	Yes			Owning tenant
campaign_id	uuid	Yes	PK FK → campaigns		Campaign
spend_date	date	Yes	PK		Day
amount	numeric(14,2)	Yes			Spend amount
currency	char(3)	Yes	FK → currencies		Currency
entry_method	text	Yes			Imported or typed
updated_at	timestamptz	Yes		now()	Last update time

- **Check:** ((amount >= (0)))
- **Check:** ((entry_method = ANY (['api', 'manual'])))

events

Events and roadshows that leads can be tagged to.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Event ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
name	text	Yes			Event name
type	text	Yes			Event type
location	text				Venue / city
starts_at	timestamptz	Yes			Start
ends_at	timestamptz	Yes			End
cost	numeric(14,2)				Total cost
currency	char(3)		FK → currencies		Cost currency
created_by	uuid		FK → users		Created by
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

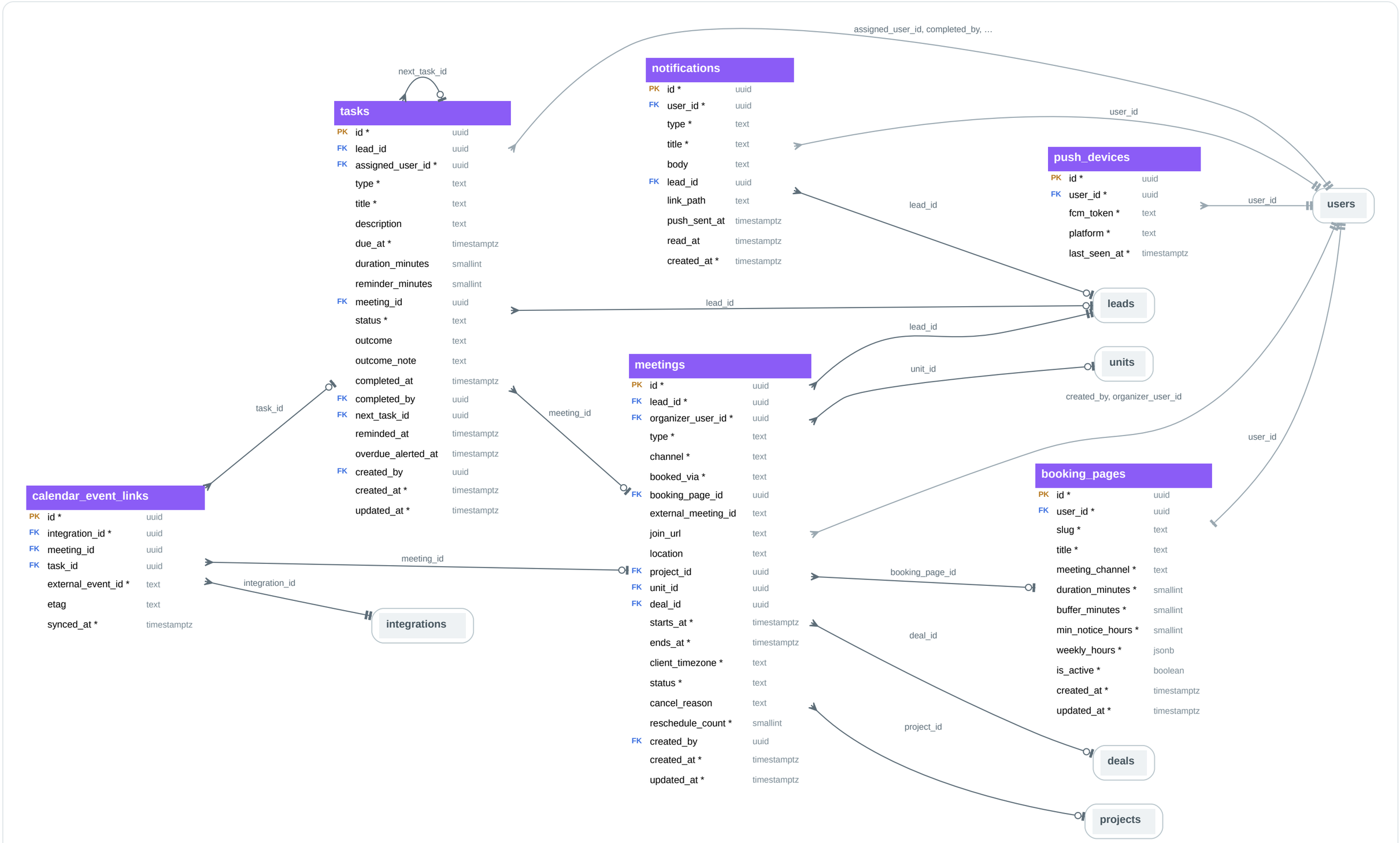
- **Unique:** (tenant_id, id)
- **Check:** (((cost IS NULL) = (currency IS NULL)))
- **Check:** ((ends_at >= starts_at))

- **Check:** `((cost >= (0)))`
- **Check:** `((type = ANY (['event', 'roadshow', 'exhibition', 'open_house'])))`

4 Tasks, meetings and notifications

In simple words: Calls, follow-ups, meetings and viewings. Each agent has a booking page so clients can book themselves. The database itself blocks double booking.

Tables: tasks · meetings · booking_pages · calendar_event_links · notifications · push_devices



ER diagram: Tasks, meetings and notifications. Grey boxes are tables from other sections. tenant_id is hidden (every table belongs to a tenant).

tasks

Calls, follow-ups, meetings, viewings and personal tasks with reminders and outcomes.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Task ID
tenant_id	uuid	Yes			Owning tenant
lead_id	uuid		FK → leads		Lead (empty for personal tasks)
assigned_user_id	uuid	Yes	FK → users		Responsible user
type	text	Yes			Task type
title	text	Yes			Title
description	text				Details
due_at	timestamptz	Yes			Due date and time
duration_minutes	smallint				Planned duration
reminder_minutes	smallint				Remind this many minutes before
meeting_id	uuid		FK → meetings		Linked meeting / viewing
status	text	Yes		'open'	Status
outcome	text				Call outcome
outcome_note	text				Outcome note
completed_at	timestamptz				Completed time
completed_by	uuid		FK → users		Completed by
next_task_id	uuid		FK → tasks		Follow-up task created on completion
reminded_at	timestamptz				Reminder sent time
overdue_alerted_at	timestamptz				Overdue alert sent time
created_by	uuid		FK → users		Created by (empty = system)
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** (tenant_id, id)
- **Check:** (((outcome IS NULL) OR (type = 'call')))
- **Check:** ((type = ANY (['call', 'follow_up', 'meeting', 'viewing', 'personal'])))
- **Check:** ((duration_minutes > 0))
- **Check:** ((reminder_minutes >= 0))
- **Check:** ((status = ANY (['open', 'completed', 'cancelled'])))
- **Check:** ((outcome = ANY (['answered', 'no_answer', 'busy', 'wrong_number', 'call_back'])))
- **Check:** (((status = 'completed') = (completed_at IS NOT NULL)))
- **Check:** (((type <> 'call') OR (status <> 'completed') OR (outcome IS NOT NULL)))
- **Check:** (((type = 'personal') OR (lead_id IS NOT NULL)))
- **Check:** ((next_task_id IS DISTINCT FROM id))
- **Index:** (tenant_id, lead_id, due_at)
- **Index:** (due_at) WHERE ((status = 'open'::text) AND (reminded_at IS NULL))
- **Index:** (tenant_id, assigned_user_id, due_at) WHERE (status = 'open'::text)

meetings

Online meetings (Google Meet / Teams) and property viewings. The database blocks double booking of an agent.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Meeting ID
tenant_id	uuid	Yes			Owning tenant
lead_id	uuid	Yes	FK → leads		Lead
organizer_user_id	uuid	Yes	FK → users		Agent attending
type	text	Yes			Meeting or property viewing
channel	text	Yes			How it happens
booked_via	text	Yes		'agent'	Booked by the agent or by the client on the booking page
booking_page_id	uuid		FK → booking_pages		Booking page used by the client
external_meeting_id	text				Google Meet / Teams meeting ID
join_url	text				Online meeting link
location	text				Address for in-person / viewing
project_id	uuid		FK → projects		Project shown
unit_id	uuid		FK → units		Unit shown
deal_id	uuid		FK → deals		Related deal
starts_at	timestamptz	Yes			Start
ends_at	timestamptz	Yes			End
client_timezone	text	Yes		'Asia/Dubai'	Client time zone for messages
status	text	Yes		'scheduled'	Status
cancel_reason	text				Cancellation reason
reschedule_count	smallint	Yes		0	Times rescheduled
created_by	uuid		FK → users		Created by (empty = client via booking page)
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** (tenant_id, id)
- **No overlap:** EXCLUDE USING gist (organizer_user_id WITH =, tstzrange(starts_at, ends_at) WITH &&) WHERE ((status = 'scheduled'::text))
- **Check:** ((ends_at > starts_at))
- **Check:** ((status = ANY (['scheduled', 'completed', 'cancelled', 'no_show'])))
- **Check:** ((type = ANY (['meeting', 'viewing'])))
- **Check:** (((channel = ANY (['in_person', 'phone'])) OR (join_url IS NOT NULL) OR (status = 'cancelled')))
- **Check:** ((channel = ANY (['google_meet', 'teams', 'in_person', 'phone'])))
- **Check:** (((booked_via = 'booking_page') = (booking_page_id IS NOT NULL)))
- **Check:** ((booked_via = ANY (['agent', 'booking_page'])))
- **Index:** (tenant_id, lead_id, starts_at)
- **Index:** (tenant_id, organizer_user_id, starts_at)

booking_pages

Each agent's public booking page (replaces Calendly). Clients pick a free slot from the agent's calendar.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Booking page ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
user_id	uuid	Yes	FK → users		Agent whose calendar is offered
slug	text	Yes			Public link name, e.g. sara-ali
title	text	Yes			Page heading
meeting_channel	text	Yes		'google_meet'	Default meeting type
duration_minutes	smallint	Yes		30	Slot length
buffer_minutes	smallint	Yes		10	Gap between meetings
min_notice_hours	smallint	Yes		2	Earliest bookable time from now
weekly_hours	jsonb	Yes		'{}'	Bookable hours per weekday, e.g. {"mon":[["09:00","17:00"]]}
is_active	boolean	Yes		true	Page on / off
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** (slug)
- **Unique:** (user_id)
- **Unique:** (tenant_id, id)
- **Check:** ((meeting_channel = ANY (['google_meet', 'teams', 'in_person', 'phone'])))
- **Check:** ((buffer_minutes >= 0))
- **Check:** ((slug ~ '^[a-z0-9-]{3,60}$'))
- **Check:** ((min_notice_hours >= 0))
- **Check:** (((duration_minutes >= 10) AND (duration_minutes <= 240)))
- **On delete:** user_id → users: cascade

calendar_event_links

Links a meeting or task to the event copy in a user's Google / Microsoft calendar.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Link ID
tenant_id	uuid	Yes			Owning tenant
integration_id	uuid	Yes	FK → integrations		User's calendar connection
meeting_id	uuid		FK → meetings		Linked meeting
task_id	uuid		FK → tasks		Linked task
external_event_id	text	Yes			Event ID in the calendar
etag	text				Version tag for change detection
synced_at	timestamptz	Yes		now()	Last sync

- **Unique:** (integration_id, external_event_id)
- **Check:** ((num_nonnulls(meeting_id, task_id) = 1))
- **On delete:** integration_id → integrations: cascade
- **On delete:** meeting_id → meetings: cascade

- **On delete:** `task_id → tasks: cascade`

notifications

In-app notifications (reminders, escalations, assignments). Also sent as push.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Notification ID
tenant_id	uuid	Yes			Owning tenant
user_id	uuid	Yes	FK → users		Recipient
type	text	Yes			e.g. lead_assigned, lead_re_enquired, lead_no_response, lead_admin_fallback, lead_shared, task_due, task_overdue, export_ready
title	text	Yes			Title
body	text				Text
lead_id	uuid		FK → leads		Related lead
link_path	text				Screen to open
push_sent_at	timestamptz				Push sent time
read_at	timestamptz				Read time
created_at	timestamptz	Yes		now()	Created time

- **Index:** `(tenant_id, user_id, created_at DESC) WHERE (read_at IS NULL)`
- **On delete:** `user_id → users: cascade`

push_devices

Phones and browsers registered for push notifications.

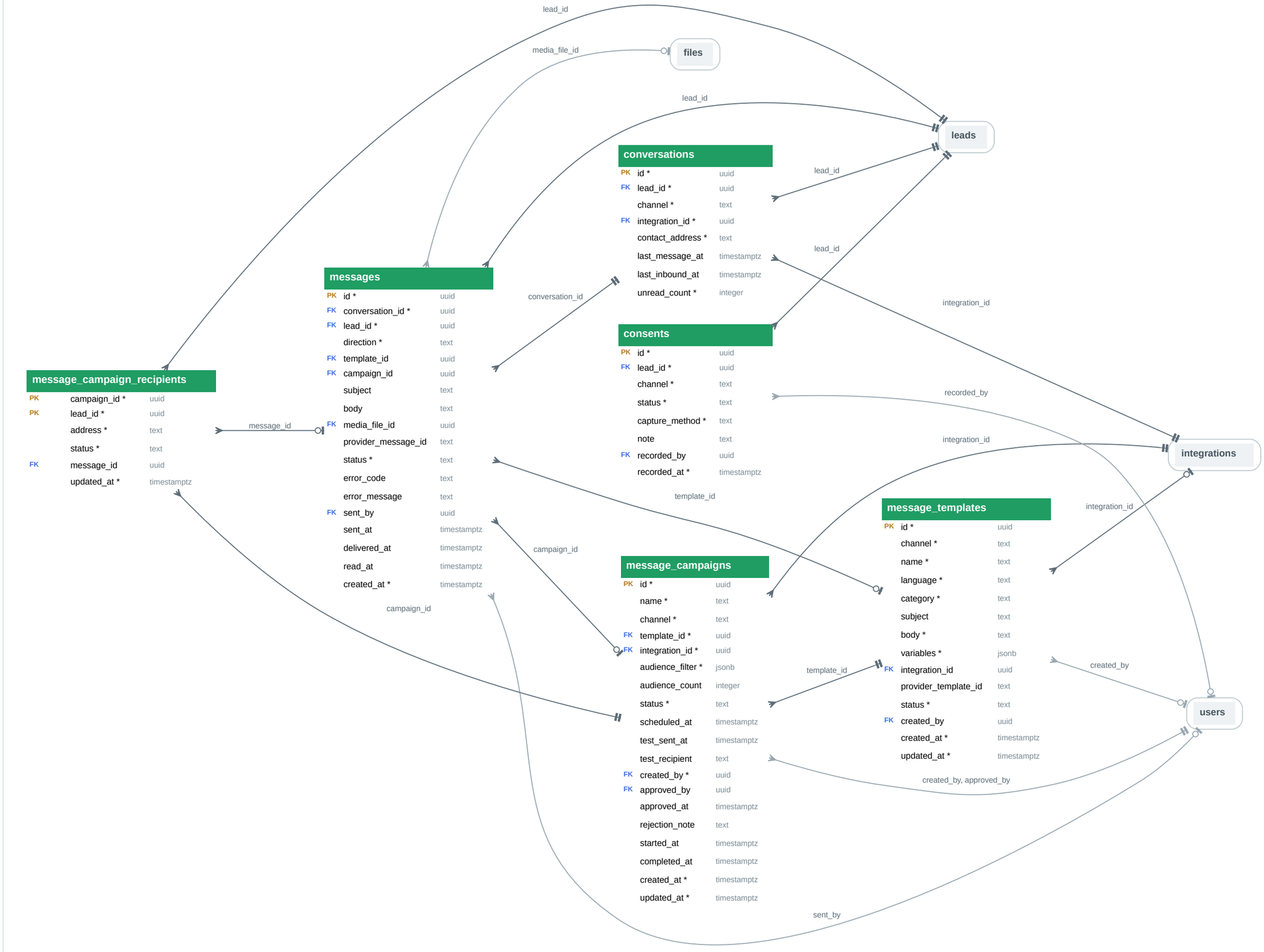
COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Device ID
tenant_id	uuid	Yes			Owning tenant
user_id	uuid	Yes	FK → users		User
fcm_token	text	Yes			Firebase token
platform	text	Yes			Device type
last_seen_at	timestamptz	Yes		now()	Last used

- **Unique:** `(fcm_token)`
- **Check:** `((platform = ANY (['android', 'ios', 'web'])))`
- **On delete:** `user_id → users: cascade`

5 WhatsApp and email

In simple words: All WhatsApp and email messages with their delivery status, approved templates, bulk campaigns and each lead's consent history.

Tables: conversations · messages · message_templates · message_campaigns · message_campaign_recipients · consents



ER diagram: WhatsApp and email. Grey boxes are tables from other sections. tenant_id is hidden (every table belongs to a tenant).

conversations

One WhatsApp or email thread per lead, channel and sending account.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Conversation ID
tenant_id	uuid	Yes			Owning tenant
lead_id	uuid	Yes	FK → leads		Lead
channel	text	Yes			Channel
integration_id	uuid	Yes	FK → integrations		Our number / mailbox
contact_address	text	Yes			Lead's phone (E.164) or email
last_message_at	timestamptz				Last message time
last_inbound_at	timestamptz				Last message from the lead (opens the 24 h WhatsApp window)
unread_count	integer	Yes		0	Unread inbound messages

- **Unique:** `(integration_id, contact_address)`
- **Unique:** `(tenant_id, id)`
- **Check:** `((channel = ANY (['whatsapp', 'email'])))`
- **Check:** `((unread_count >= 0))`
- **Index:** `(tenant_id, lead_id)`

messages

Every WhatsApp and email message, in and out, with delivery status.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Message ID
tenant_id	uuid	Yes			Owning tenant
conversation_id	uuid	Yes	FK → conversations		Conversation
lead_id	uuid	Yes	FK → leads		Lead (copied for fast timeline queries)
direction	text	Yes			In or out
template_id	uuid		FK → message_templates		Template used
campaign_id	uuid		FK → message_campaigns		Bulk campaign it belongs to
subject	text				Email subject
body	text				Text body
media_file_id	uuid		FK → files		Attachment / media
provider_message_id	text				Message ID at the provider
status	text	Yes			Delivery status (queued = attempted)
error_code	text				Provider error code
error_message	text				Provider error text
sent_by	uuid		FK → users		Sending user (empty = system / campaign)
sent_at	timestamptz				Sent time
delivered_at	timestamptz				Delivered time

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
read_at	timestamptz				Read time
created_at	timestamptz	Yes		now()	Created time

- **Unique:** (tenant_id, id)
- **Check:** ((direction = ANY (['inbound', 'outbound'])))
- **Check:** (((body IS NOT NULL) OR (media_file_id IS NOT NULL) OR (template_id IS NOT NULL)))
- **Check:** (((direction = 'inbound') = (status = 'received')))
- **Check:** ((status = ANY (['queued', 'sent', 'delivered', 'read', 'failed', 'received'])))
- **Index:** (tenant_id, campaign_id) WHERE (campaign_id IS NOT NULL)
- **Index:** (tenant_id, conversation_id, created_at)
- **Index:** (tenant_id, lead_id, created_at)
- **Index:** Unique (conversation_id, provider_message_id) WHERE (provider_message_id IS NOT NULL)

message_templates

Approved WhatsApp templates and email templates.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Template ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
channel	text	Yes			Channel
name	text	Yes			Template name (provider name for WhatsApp)
language	text	Yes		'en'	Language code
category	text	Yes			Meta category / email type
subject	text				Email subject
body	text	Yes			Body with {{placeholders}}
variables	jsonb	Yes		'[]'	Placeholder definitions
integration_id	uuid		FK → integrations		WhatsApp number / provider it is registered with
provider_template_id	text				Template ID at the provider
status	text	Yes		'draft'	Approval status
created_by	uuid		FK → users		Created by
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** (tenant_id, id)
- **Unique:** (tenant_id, channel, name, language)
- **Check:** ((category = ANY (['marketing', 'utility', 'authentication', 'transactional'])))
- **Check:** (((channel <> 'email') OR (subject IS NOT NULL)))
- **Check:** ((channel = ANY (['whatsapp', 'email'])))
- **Check:** ((status = ANY (['draft', 'pending', 'approved', 'rejected', 'disabled'])))

message_campaigns

Bulk WhatsApp / email campaigns with test, approval and schedule.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Campaign ID
tenant_id	uuid	Yes			Owning tenant
name	text	Yes			Campaign name
channel	text	Yes			Channel
template_id	uuid	Yes	FK → message_templates		Template
integration_id	uuid	Yes	FK → integrations		Sending number / mailbox
audience_filter	jsonb	Yes			Lead filter that defines the audience
audience_count	integer				Recipients after opt-out exclusion
status	text	Yes		'draft'	Status
scheduled_at	timestamptz				Planned send time
test_sent_at	timestamptz				Test message sent time
test_recipient	text				Test phone / email
created_by	uuid	Yes	FK → users		Created by
approved_by	uuid		FK → users		Approved / rejected by
approved_at	timestamptz				Approval time
rejection_note	text				Rejection reason
started_at	timestamptz				Sending started
completed_at	timestamptz				Sending finished
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** (tenant_id, id)
- **Check:** ((channel = ANY (['whatsapp', 'email'])))
- **Check:** (((approved_by IS DISTINCT FROM created_by) OR (approved_by IS NULL)))
- **Check:** (((status <> 'rejected') OR (rejection_note IS NOT NULL)))
- **Check:** (((status <> ALL (['approved', 'sending', 'completed'])) OR (approved_by IS NOT NULL)))
- **Check:** ((status = ANY (['draft', 'pending_approval', 'approved', 'rejected', 'sending', 'completed', 'cancelled'])))

message_campaign_recipients

Each lead in a bulk campaign and what happened to its message.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
tenant_id	uuid	Yes			Owning tenant
campaign_id	uuid	Yes	PK FK → message_campaigns		Campaign
lead_id	uuid	Yes	PK FK → leads		Recipient lead
address	text	Yes			Phone or email used
status	text	Yes		'pending'	Result
message_id	uuid		FK → messages		Message sent

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
updated_at	timestamptz	Yes		now()	Last status change

- **Check:** ((status = ANY (['pending', 'skipped_opted_out', 'sent', 'delivered', 'failed', 'replied', 'opted_out'])))
- **On delete:** campaign_id → message_campaigns: cascade

consents

Consent and opt-out history per lead and channel. Latest row wins. Never changed after saving.

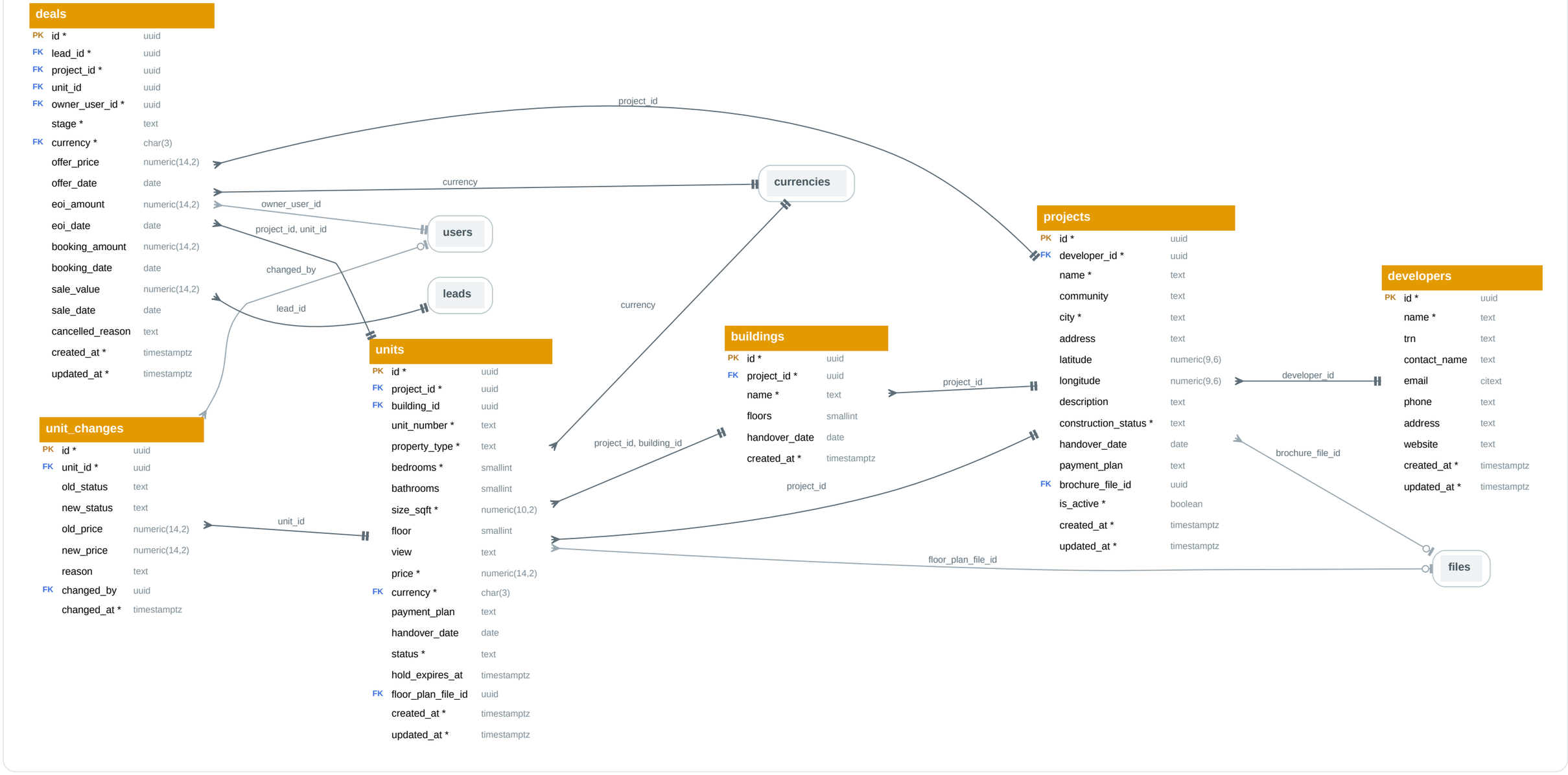
COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Consent record ID
tenant_id	uuid	Yes			Owning tenant
lead_id	uuid	Yes	FK → leads		Lead
channel	text	Yes			Channel
status	text	Yes			Consent given or withdrawn
capture_method	text	Yes			How it was captured
note	text				Evidence / comment
recorded_by	uuid		FK → users		Recorded by (empty = system)
recorded_at	timestamptz	Yes		now()	Time recorded

- **Check:** ((status = ANY (['opted_in', 'opted_out'])))
- **Check:** ((channel = ANY (['whatsapp', 'email', 'sms'])))
- **Check:** ((capture_method = ANY (['form', 'import', 'reply', 'unsubscribe_link', 'manual', 'provider'])))
- **Index:** (tenant_id, lead_id, channel, recorded_at DESC)

6 Projects, units and deals

In simple words: Developers, projects, buildings and units for sale, with every price and status change kept. A deal links a buyer to one unit; a unit can only be booked once.

Tables: developers · projects · buildings · units · unit_changes · deals



ER diagram: Projects, units and deals. Grey boxes are tables from other sections. tenant_id is hidden (every table belongs to a tenant).

developers

Property developers (e.g. Emaar, Sobha).

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Developer ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
name	text	Yes			Developer name
trn	text				VAT registration number
contact_name	text				Main contact
email	citext				Email
phone	text				Phone (E.164)
address	text				Address
website	text				Website
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** (tenant_id, id)
- **Unique:** (tenant_id, name)

projects

Real estate projects sold by the brokerage.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Project ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
developer_id	uuid	Yes	FK → developers		Developer
name	text	Yes			Project name
community	text				Community / area, e.g. Dubai Hills
city	text	Yes		'Dubai'	City
address	text				Address
latitude	numeric(9,6)				Map latitude
longitude	numeric(9,6)				Map longitude
description	text				Description
construction_status	text	Yes		'off_plan'	Build status
handover_date	date				Expected handover
payment_plan	text				Payment plan, e.g. 80/20
brochure_file_id	uuid		FK → files		Project brochure
is_active	boolean	Yes		true	Active flag
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** `(tenant_id, developer_id, name)`
- **Unique:** `(tenant_id, id)`
- **Check:** `(((latitude >= ('-90'::integer)) AND (latitude <= (90))))`
- **Check:** `((construction_status = ANY (['off_plan', 'under_construction', 'ready'])))`
- **Check:** `(((longitude >= ('-180'::integer)) AND (longitude <= (180))))`

buildings

Buildings or phases within a project.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Building ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
project_id	uuid	Yes	FK → projects		Project
name	text	Yes			Building name, e.g. Tower A
floors	smallint				Number of floors
handover_date	date				Building handover date
created_at	timestamptz	Yes		now()	Created time

- **Unique:** `(project_id, name)`
- **Unique:** `(tenant_id, project_id, id)`
- **Unique:** `(tenant_id, id)`
- **Check:** `((floors > 0))`

units

Individual units for sale. Status: available, on hold, booked, sold.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Unit ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
project_id	uuid	Yes	FK → buildings		Project
building_id	uuid		FK → buildings		Building (empty for villas/plots)
unit_number	text	Yes			Unit number, e.g. 1204
property_type	text	Yes			Property type
bedrooms	smallint	Yes			Bedrooms (0 = studio)
bathrooms	smallint				Bathrooms
size_sqft	numeric(10,2)	Yes			Size in square feet
floor	smallint				Floor number
view	text				View, e.g. Sea
price	numeric(14,2)	Yes			Current price
currency	char(3)	Yes	FK → currencies	'AED'	Price currency
payment_plan	text				Unit payment plan (overrides project)

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
handover_date	date				Unit handover date
status	text	Yes		'available'	Availability
hold_expires_at	timestamptz				When an On Hold status ends
floor_plan_file_id	uuid		FK → files		Floor plan file
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** Unique (empty values count as equal) (project_id, building_id, unit_number)
- **Unique:** (tenant_id, id)
- **Unique:** (tenant_id, project_id, id)
- **Check:** ((size_sqft > (0)))
- **Check:** ((property_type = ANY (['apartment', 'villa', 'townhouse', 'penthouse', 'duplex', 'office', 'retail', 'plot'])))
- **Check:** (((bedrooms >= 0) AND (bedrooms <= 20)))
- **Check:** (((bathrooms >= 0) AND (bathrooms <= 20)))
- **Check:** ((price >= (0)))
- **Check:** ((status = ANY (['available', 'on_hold', 'booked', 'sold'])))
- **Check:** (((status = 'on_hold') OR (hold_expires_at IS NULL)))
- **Index:** (tenant_id, project_id, status)
- **Index:** (tenant_id, property_type, bedrooms, price)

unit_changes

History of every unit price and status change.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Change ID
tenant_id	uuid	Yes			Owning tenant
unit_id	uuid	Yes	FK → units		Unit
old_status	text				Status before
new_status	text				Status after
old_price	numeric(14,2)				Price before
new_price	numeric(14,2)				Price after
reason	text				Why it changed
changed_by	uuid		FK → users		Changed by (empty = system)
changed_at	timestamptz	Yes		now()	Change time

- **Check:** (((old_status IS DISTINCT FROM new_status) OR (old_price IS DISTINCT FROM new_price)))
- **Index:** (tenant_id, unit_id, changed_at)

deals

A lead's purchase of one unit: offer, EOI, booking and sale.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Deal ID

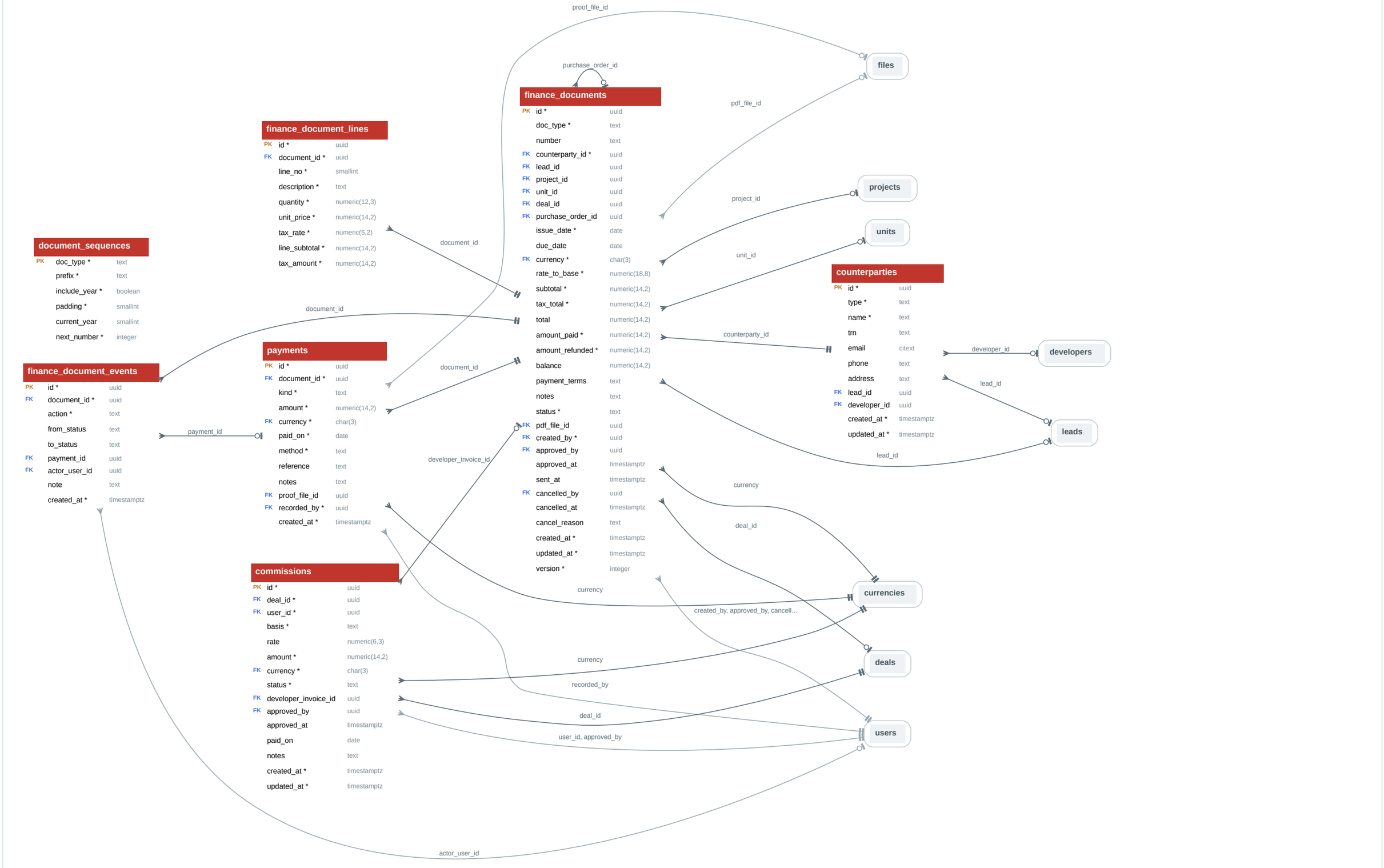
COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
tenant_id	uuid	Yes			Owning tenant
lead_id	uuid	Yes	FK → leads		Buyer lead
project_id	uuid	Yes	FK → units		Project
unit_id	uuid		FK → units		Chosen unit (must belong to the project)
owner_user_id	uuid	Yes	FK → users		Agent handling the deal
stage	text	Yes		'offer'	Deal stage
currency	char(3)	Yes	FK → currencies	'AED'	Deal currency
offer_price	numeric(14,2)				Offered price
offer_date	date				Offer date
eoi_amount	numeric(14,2)				EOI deposit
eoi_date	date				EOI date
booking_amount	numeric(14,2)				Booking amount
booking_date	date				Booking date
sale_value	numeric(14,2)				Final sale value
sale_date	date				Sale date
cancelled_reason	text				Why the deal was cancelled
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** (tenant_id, id)
- **Check:** ((eoi_amount >= (0)))
- **Check:** (((stage <> ALL (['booking', 'sold'])) OR ((unit_id IS NOT NULL) AND (booking_date IS NOT NULL))))
- **Check:** (((stage <> 'sold') OR ((sale_value IS NOT NULL) AND (sale_date IS NOT NULL))))
- **Check:** (((stage <> 'cancelled') OR (cancelled_reason IS NOT NULL)))
- **Check:** ((stage = ANY (['offer', 'eoi', 'booking', 'sold', 'cancelled'])))
- **Check:** ((offer_price >= (0)))
- **Check:** ((sale_value >= (0)))
- **Check:** ((booking_amount >= (0)))
- **Check:** (((stage <> 'eoi') OR (eoi_date IS NOT NULL)))
- **Index:** (tenant_id, lead_id)
- **Index:** Unique (unit_id) WHERE (stage = ANY (ARRAY['booking'::text, 'sold'::text]))
- **Index:** (tenant_id, project_id, stage)

7 Finance

In simple words: Invoices and purchase orders with lines, VAT, payments and full approval history. After approval, amounts are locked by the database.

Tables: finance_documents · finance_document_lines · payments · finance_document_events · counterparties · document_sequences · commissions



ER diagram: Finance. Grey boxes are tables from other sections. tenant_id is hidden (every table belongs to a tenant).

finance_documents

Sales invoices, purchase invoices and purchase orders.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Document ID
tenant_id	uuid	Yes			Owning tenant
doc_type	text	Yes			Type
number	text				Document number (given on approval; empty while Draft)
counterparty_id	uuid	Yes	FK → counterparties		Client, developer or vendor
lead_id	uuid		FK → leads		Linked lead
project_id	uuid		FK → projects		Linked project
unit_id	uuid		FK → units		Linked unit
deal_id	uuid		FK → deals		Linked deal
purchase_order_id	uuid		FK → finance_documents		PO this purchase invoice settles
issue_date	date	Yes			Issue date
due_date	date				Due date
currency	char(3)	Yes	FK → currencies		Currency
rate_to_base	numeric(18,8)	Yes		1	Exchange rate to base currency at issue
subtotal	numeric(14,2)	Yes		0	Sum of lines before tax
tax_total	numeric(14,2)	Yes		0	Sum of line tax
total	numeric(14,2)			calculated: (subtotal + tax_total)	Total incl. tax
amount_paid	numeric(14,2)	Yes		0	Payments received / made
amount_refunded	numeric(14,2)	Yes		0	Refunds
balance	numeric(14,2)			calculated: (((subtotal + tax_total) - amount_paid) + amount_refunded)	Outstanding amount
payment_terms	text				Payment terms
notes	text				Notes printed on the document
status	text	Yes		'draft'	Status
pdf_file_id	uuid		FK → files		Generated PDF
created_by	uuid	Yes	FK → users		Created by
approved_by	uuid		FK → users		Approved by
approved_at	timestamptz				Approval time
sent_at	timestamptz				Sent time
cancelled_by	uuid		FK → users		Cancelled by
cancelled_at	timestamptz				Cancellation time
cancel_reason	text				Cancellation reason
created_at	timestamptz	Yes		now()	Created time

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
updated_at	timestamptz	Yes		now()	Last update time
version	integer	Yes		1	Optimistic lock counter

- **Unique:** (tenant_id, id)
- **Unique:** (tenant_id, doc_type, number)
- **Check:** ((rate_to_base > (0)))
- **Check:** ((doc_type = ANY (['sales_invoice', 'purchase_invoice', 'purchase_order'])))
- **Check:** ((subtotal >= (0)))
- **Check:** ((tax_total >= (0)))
- **Check:** ((amount_paid >= (0)))
- **Check:** ((amount_refunded >= (0)))
- **Check:** ((status = ANY (['draft', 'approved', 'sent', 'partially_paid', 'paid', 'overdue', 'cancelled', 'refunded'])))
- **Check:** (((due_date IS NULL) OR (due_date >= issue_date)))
- **Check:** (((doc_type <> 'purchase_order') OR (status = ANY (['draft', 'approved', 'sent', 'cancelled']))))
- **Check:** (((status = 'draft') OR ((number IS NOT NULL) AND (approved_by IS NOT NULL) AND (approved_at IS NOT NULL))))
- **Check:** (((status = 'cancelled') = (cancelled_at IS NOT NULL)))
- **Check:** (((status <> 'cancelled') OR (cancel_reason IS NOT NULL)))
- **Check:** (((purchase_order_id IS NULL) OR (doc_type = 'purchase_invoice')))
- **Check:** ((purchase_order_id IS DISTINCT FROM id))
- **Index:** (tenant_id, deal_id) WHERE (deal_id IS NOT NULL)
- **Index:** (tenant_id, lead_id) WHERE (lead_id IS NOT NULL)
- **Index:** (tenant_id, doc_type, status, due_date)

finance_document_lines

Line items of a financial document. Can only change while the document is Draft.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Line ID
tenant_id	uuid	Yes			Owning tenant
document_id	uuid	Yes	FK → finance_documents		Document
line_no	smallint	Yes			Line order
description	text	Yes			Description
quantity	numeric(12,3)	Yes		1	Quantity
unit_price	numeric(14,2)	Yes			Price per unit before tax
tax_rate	numeric(5,2)	Yes		5.00	VAT % (UAE standard 5)
line_subtotal	numeric(14,2)	Yes			quantity × unit_price, rounded
tax_amount	numeric(14,2)	Yes			line_subtotal × tax_rate, rounded

- **Unique:** (document_id, line_no)
- **Check:** ((line_subtotal >= (0)))
- **Check:** ((line_no > 0))
- **Check:** ((quantity > (0)))
- **Check:** ((unit_price >= (0)))
- **Check:** ((tax_amount >= (0)))
- **Check:** (((tax_rate >= (0)) AND (tax_rate <= (100))))
- **Index:** (tenant_id, document_id)

- **On delete:** `document_id → finance_documents: cascade`

payments

Payments and refunds recorded against a document.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Payment ID
tenant_id	uuid	Yes			Owning tenant
document_id	uuid	Yes	FK → finance_documents		Document
kind	text	Yes			Payment or refund
amount	numeric(14,2)	Yes			Amount (always positive)
currency	char(3)	Yes	FK → currencies		Currency (same as document)
paid_on	date	Yes			Date of payment
method	text	Yes			Method
reference	text				Bank / cheque reference
notes	text				Notes
proof_file_id	uuid		FK → files		Receipt / proof
recorded_by	uuid	Yes	FK → users		Recorded by
created_at	timestamptz	Yes		now()	Recorded time

- **Unique:** `(tenant_id, id)`
- **Check:** `((kind = ANY (['payment', 'refund'])))`
- **Check:** `((method = ANY (['bank_transfer', 'cheque', 'card', 'cash', 'other'])))`
- **Check:** `((amount > (0)))`
- **Index:** `(tenant_id, document_id)`

finance_document_events

Approval and status history of each document. Never changed after saving.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Event ID
tenant_id	uuid	Yes			Owning tenant
document_id	uuid	Yes	FK → finance_documents		Document
action	text	Yes			What happened
from_status	text				Status before
to_status	text				Status after
payment_id	uuid		FK → payments		Related payment
actor_user_id	uuid		FK → users		Who (empty = system)
note	text				Comment
created_at	timestamptz	Yes		now()	Time

- **Check:** `((action = ANY (['created', 'edited', 'approved', 'sent', 'payment_recorded', 'refund_recorded', 'overdue', 'cancelled', 'pdf_generated'])))`
- **Index:** `(tenant_id, document_id, created_at)`

counterparties

Who a financial document is with: client, developer or vendor.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Counterparty ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
type	text	Yes			Kind
name	text	Yes			Name on documents
trn	text				VAT registration number
email	citext				Billing email
phone	text				Phone
address	text				Billing address
lead_id	uuid		FK → leads		Linked lead (clients)
developer_id	uuid		FK → developers		Linked developer (developers)
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** `(tenant_id, id)`
- **Check:** `(((type = 'developer') = (developer_id IS NOT NULL)))`
- **Check:** `((type = ANY (['client', 'developer', 'vendor'])))`
- **Check:** `(((type = 'client') OR (lead_id IS NULL)))`

document_sequences

Next document number per type (e.g. INV-2026-0042).

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
tenant_id	uuid	Yes	PK FK → tenants		Owning tenant
doc_type	text	Yes	PK		Document type
prefix	text	Yes			Prefix, e.g. INV-
include_year	boolean	Yes		true	Add the year and restart each year
padding	smallint	Yes		4	Digits, e.g. 4 = 0042
current_year	smallint				Year of the counter (when include_year)
next_number	integer	Yes		1	Next number to issue

- **Check:** `((doc_type = ANY (['sales_invoice', 'purchase_invoice', 'purchase_order'])))`
- **Check:** `((next_number > 0))`
- **Check:** `(((padding >= 1) AND (padding <= 10)))`

commissions

Agent commission per deal, with the developer commission invoice.

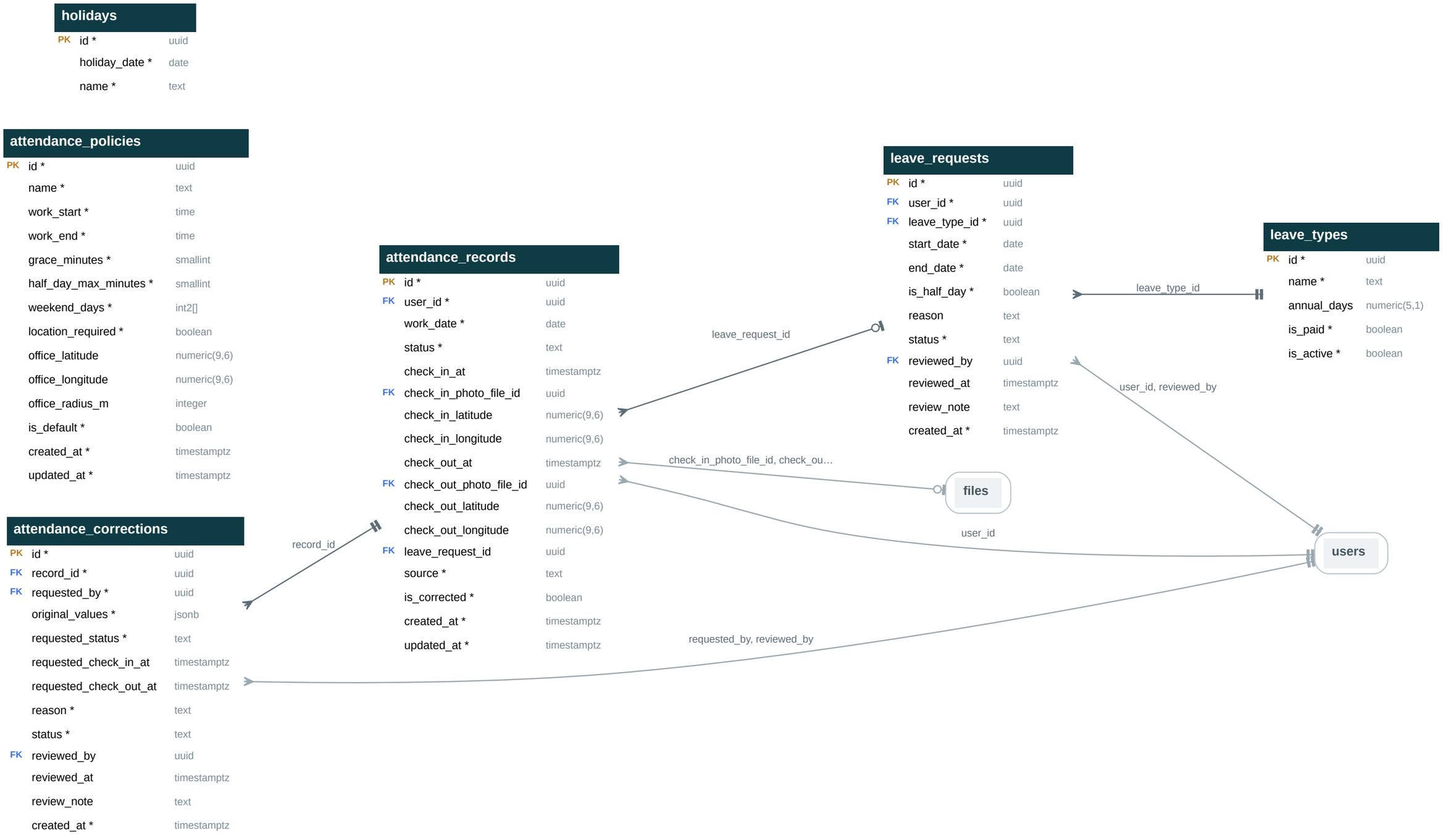
COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Commission ID
tenant_id	uuid	Yes			Owning tenant
deal_id	uuid	Yes	FK → deals		Deal
user_id	uuid	Yes	FK → users		Agent earning it
basis	text	Yes			Calculation basis
rate	numeric(6,3)				% of sale value (percentage basis)
amount	numeric(14,2)	Yes			Commission amount
currency	char(3)	Yes	FK → currencies		Currency
status	text	Yes		'pending'	Status
developer_invoice_id	uuid		FK → finance_documents		Sales invoice to the developer
approved_by	uuid		FK → users		Approved by
approved_at	timestamptz				Approval time
paid_on	date				Paid date
notes	text				Notes
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** (deal_id, user_id)
- **Unique:** (tenant_id, id)
- **Check:** ((amount >= (0)))
- **Check:** (((status = 'paid') = (paid_on IS NOT NULL)))
- **Check:** (((status <> ALL (['approved', 'paid'])) OR (approved_by IS NOT NULL)))
- **Check:** (((basis = 'percentage') = (rate IS NOT NULL)))
- **Check:** ((status = ANY (['pending', 'approved', 'paid', 'cancelled'])))
- **Check:** (((rate >= (0)) AND (rate <= (100))))
- **Check:** ((basis = ANY (['percentage', 'fixed'])))
- **Index:** (tenant_id, user_id, status)

8 Attendance and leave

In simple words: One attendance row per person per day with check-in photo. Corrections keep the original values. Approved leave cannot overlap and blocks new leads.

Tables: attendance_records · attendance_corrections · leave_requests · leave_types · attendance_policies · holidays



ER diagram: Attendance and leave. Grey boxes are tables from other sections. tenant_id is hidden (every table belongs to a tenant).

attendance_records

One row per employee per working day, with check-in/out photo, time and location.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Record ID
tenant_id	uuid	Yes			Owning tenant
user_id	uuid	Yes	FK → users		Employee
work_date	date	Yes			Working day
status	text	Yes			Day status
check_in_at	timestamptz				Check-in time
check_in_photo_file_id	uuid		FK → files		Check-in live photo
check_in_latitude	numeric(9,6)				Check-in location (if enabled)
check_in_longitude	numeric(9,6)				Check-in location (if enabled)
check_out_at	timestamptz				Check-out time
check_out_photo_file_id	uuid		FK → files		Check-out live photo
check_out_latitude	numeric(9,6)				Check-out location (if enabled)
check_out_longitude	numeric(9,6)				Check-out location (if enabled)
leave_request_id	uuid		FK → leave_requests		Approved leave covering this day
source	text	Yes		'app'	How the row was last set
is_corrected	boolean	Yes		false	Changed by an approved correction
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** (user_id, work_date)
- **Unique:** (tenant_id, id)
- **Check:** ((source = ANY (['app', 'system', 'correction'])))
- **Check:** (((status <> 'leave') OR (leave_request_id IS NOT NULL)))
- **Check:** (((source <> 'app') OR (check_in_at IS NULL) OR (check_in_photo_file_id IS NOT NULL)))
- **Check:** (((check_out_at IS NULL) OR ((check_in_at IS NOT NULL) AND (check_out_at > check_in_at))))
- **Check:** ((status = ANY (['present', 'late', 'absent', 'leave', 'half_day', 'wfh'])))
- **Index:** (tenant_id, work_date)

attendance_corrections

Correction requests. The original values are kept in original_values; the record keeps the corrected values.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Correction ID
tenant_id	uuid	Yes			Owning tenant
record_id	uuid	Yes	FK → attendance_records		Attendance record
requested_by	uuid	Yes	FK → users		Employee asking
original_values	jsonb	Yes			Copy of the record before correction
requested_status	text	Yes			Requested status
requested_check_in_at	timestamptz				Requested check-in time

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
requested_check_out_at	timestamptz				Requested check-out time
reason	text	Yes			Why
status	text	Yes		'pending'	Review status
reviewed_by	uuid		FK → users		Manager who reviewed
reviewed_at	timestamptz				Review time
review_note	text				Manager comment
created_at	timestamptz	Yes		now()	Requested time

- **Check:** ((status = ANY (['pending', 'approved', 'rejected'])))
- **Check:** (((status = 'pending') = (reviewed_at IS NULL)))
- **Check:** (((status = 'pending') OR (reviewed_by IS NOT NULL)))
- **Check:** (((reviewed_by IS DISTINCT FROM requested_by) OR (reviewed_by IS NULL)))
- **Check:** ((requested_status = ANY (['present', 'late', 'absent', 'leave', 'half_day', 'wfh'])))
- **Index:** (tenant_id, created_at) WHERE (status = 'pending'::text)

leave_requests

Leave requests and approvals. Approved leave makes the user unavailable for new leads.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Request ID
tenant_id	uuid	Yes			Owning tenant
user_id	uuid	Yes	FK → users		Employee
leave_type_id	uuid	Yes	FK → leave_types		Leave type
start_date	date	Yes			First day
end_date	date	Yes			Last day
is_half_day	boolean	Yes		false	Half-day leave (start_date only)
reason	text				Reason
status	text	Yes		'pending'	Status
reviewed_by	uuid		FK → users		Approved / rejected by
reviewed_at	timestamptz				Review time
review_note	text				Reviewer comment
created_at	timestamptz	Yes		now()	Requested time

- **Unique:** (tenant_id, id)
- **No overlap:** EXCLUDE USING gist (user_id WITH =, daterange(start_date, end_date, '[]'::text) WITH &&) WHERE ((status = 'approved'::text))
- **Check:** (((status <> ALL (['approved', 'rejected'])) OR (reviewed_by IS NOT NULL)))
- **Check:** (((NOT is_half_day) OR (end_date = start_date)))
- **Check:** ((end_date >= start_date))
- **Check:** ((status = ANY (['pending', 'approved', 'rejected', 'cancelled'])))
- **Check:** (((reviewed_by IS DISTINCT FROM user_id) OR (reviewed_by IS NULL)))
- **Index:** (tenant_id, user_id, start_date)

leave_types

Kinds of leave and yearly allowance.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Leave type ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
name	text	Yes			e.g. Annual, Sick
annual_days	numeric(5,1)				Allowance per year (empty = unlimited)
is_paid	boolean	Yes		true	Paid leave
is_active	boolean	Yes		true	Active flag

- **Unique:** (tenant_id, name)
- **Unique:** (tenant_id, id)
- **Check:** ((annual_days >= (0)))

attendance_policies

Office hours, grace time, weekends and location rules.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Policy ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
name	text	Yes			Policy name
work_start	time	Yes			Office start, e.g. 09:00
work_end	time	Yes			Office end, e.g. 18:00
grace_minutes	smallint	Yes		10	Late after start + grace
half_day_max_minutes	smallint	Yes		300	Worked minutes at or below this = Half Day
weekend_days	int2[]	Yes		'{0,6}'	Weekend days (0 = Sunday … 6 = Saturday)
location_required	boolean	Yes		false	Record location at check-in
office_latitude	numeric(9,6)				Office location
office_longitude	numeric(9,6)				Office location
office_radius_m	integer				Allowed distance from office
is_default	boolean	Yes		false	Used when a user has no override
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** (tenant_id, name)
- **Unique:** (tenant_id, id)
- **Check:** ((office_radius_m > 0))
- **Check:** ((weekend_days <@ '{0,1,2,3,4,5,6}'))
- **Check:** ((work_end > work_start))
- **Check:** ((half_day_max_minutes > 0))
- **Check:** ((grace_minutes >= 0))
- **Index:** Unique (tenant_id) WHERE is_default

holidays

Public and company holidays.

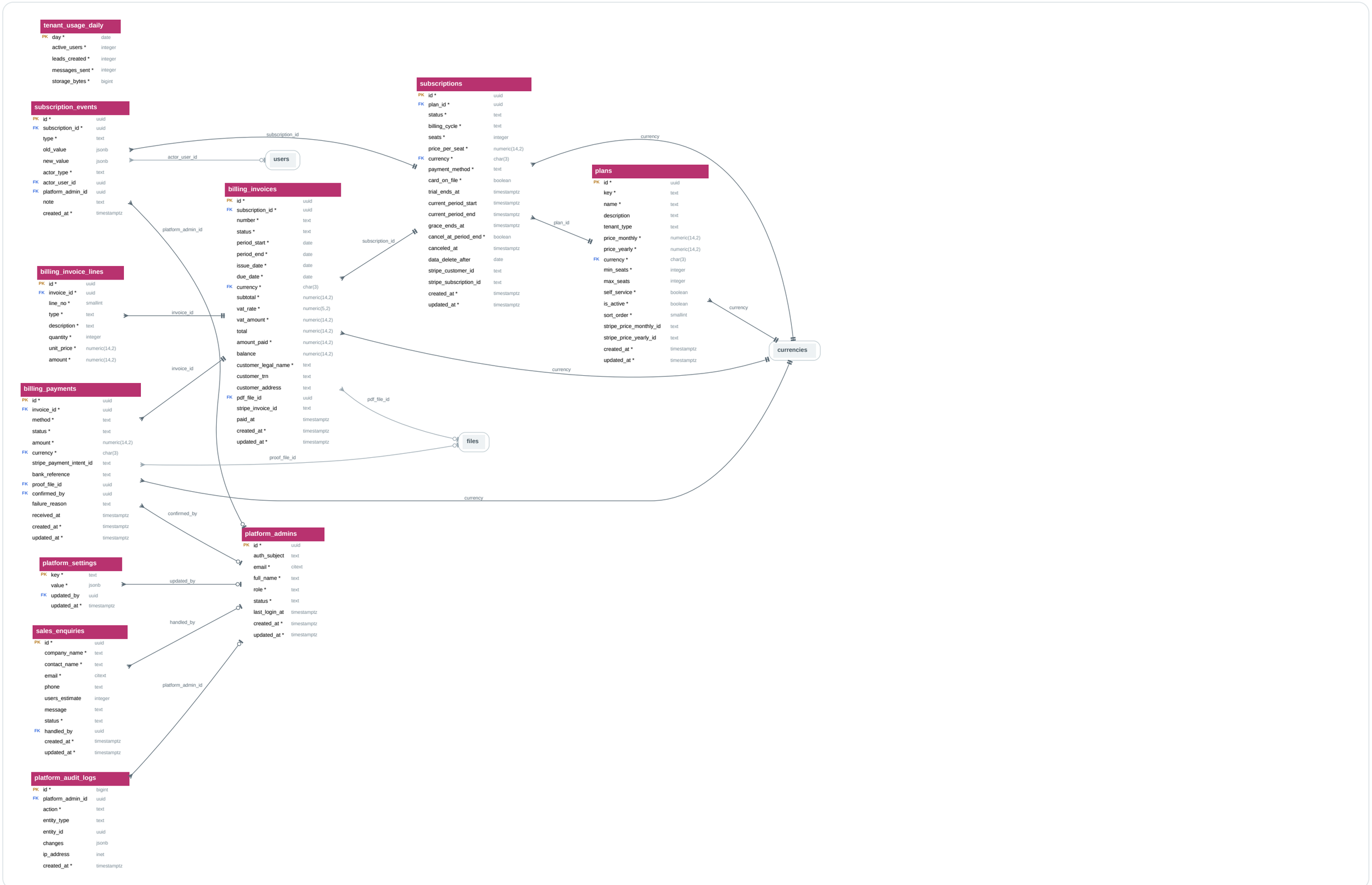
COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Holiday ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
holiday_date	date	Yes			Date
name	text	Yes			Name

- **Unique:** (tenant_id, holiday_date)

9 Subscriptions, billing and owner console

In simple words: How we sell the CRM. Each customer company has one subscription: plan, number of paid users (seats), monthly or yearly, card or bank transfer. We send VAT invoices; Stripe confirms card payments and the platform team confirms bank transfers. If payment fails, the company gets 14 days, then becomes read-only; data is never deleted while unpaid. The owner console is only for the platform team.

Tables: plans · subscriptions · subscription_events · billing_invoices · billing_invoice_lines · billing_payments · tenant_usage_daily · platform_admins · platform_settings · sales_enquiries · platform_audit_logs



ER diagram: Subscriptions, billing and owner console. Grey boxes are tables from other sections. tenant_id is hidden (every table belongs to a tenant).

plans

Subscription plans we sell (Solo, Team, Enterprise) and their prices.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Plan ID
key	text	Yes			Stable key, e.g. team
name	text	Yes			Display name
description	text				Shown on the pricing page
tenant_type	text				Who may choose it (empty = both)
price_monthly	numeric(14,2)	Yes			Price per user per month, excluding VAT
price_yearly	numeric(14,2)	Yes			Price per user per year, excluding VAT
currency	char(3)	Yes	FK → currencies	'AED'	Price currency
min_seats	integer	Yes		1	Minimum users
max_seats	integer				Maximum users (empty = no limit)
self_service	boolean	Yes		true	Can be bought online (false = through sales, e.g. Enterprise)
is_active	boolean	Yes		true	Offered to new customers
sort_order	smallint	Yes		0	Order on the pricing page
stripe_price_monthly_id	text				Stripe price ID (monthly)
stripe_price_yearly_id	text				Stripe price ID (yearly)
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** (key)
- **Check:** (((max_seats IS NULL) OR (max_seats >= min_seats)))
- **Check:** ((price_yearly >= (0)))
- **Check:** ((tenant_type = ANY (['brokerage', 'individual'])))
- **Check:** ((price_monthly >= (0)))
- **Check:** ((key ~ '^[a-z_]+$'))
- **Check:** ((min_seats >= 1))

subscriptions

Each tenant's current plan, seats, billing cycle and billing status (one per tenant).

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Subscription ID
tenant_id	uuid	Yes	FK → tenants		Tenant (one subscription each)
plan_id	uuid	Yes	FK → plans		Plan
status	text	Yes		'trialing'	trialing, active, past_due (grace), read_only (unpaid after grace), canceled
billing_cycle	text	Yes		'monthly'	Billing cycle
seats	integer	Yes			Paid users (active users cannot exceed this)
price_per_seat	numeric(14,2)	Yes			Agreed price per user per cycle, excluding VAT (custom for Enterprise)
currency	char(3)	Yes	FK → currencies	'AED'	Billing currency

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
payment_method	text	Yes		'card'	How invoices are paid
card_on_file	boolean	Yes		false	A card is saved at Stripe (required to start a trial)
trial_ends_at	timestamptz				End of the 14-day trial
current_period_start	timestamptz				Current paid period start
current_period_end	timestamptz				Current paid period end (next renewal)
grace_ends_at	timestamptz				When past_due becomes read_only (payment failed + 14 days)
cancel_at_period_end	boolean	Yes		false	Customer cancelled; ends at period end
canceled_at	timestamptz				When the subscription ended
data_delete_after	date				Tenant data deleted after this date (canceled + 90 days)
stripe_customer_id	text				Stripe customer
stripe_subscription_id	text				Stripe subscription
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** (tenant_id, id)
- **Unique:** (stripe_subscription_id)
- **Unique:** (stripe_customer_id)
- **Unique:** (tenant_id)
- **Check:** ((price_per_seat >= (0)))
- **Check:** ((payment_method = ANY (['card', 'bank_transfer'])))
- **Check:** ((status = ANY (['trialing', 'active', 'past_due', 'read_only', 'canceled'])))
- **Check:** (((status <> 'trialing') OR ((trial_ends_at IS NOT NULL) AND card_on_file)))
- **Check:** (((status <> 'past_due') OR (grace_ends_at IS NOT NULL)))
- **Check:** (((status = 'canceled') = (canceled_at IS NOT NULL)))
- **Check:** (((status <> 'canceled') OR (data_delete_after IS NOT NULL)))
- **Check:** (((current_period_end IS NULL) OR (current_period_end > current_period_start)))
- **Check:** ((billing_cycle = ANY (['monthly', 'yearly'])))
- **Check:** ((seats >= 1))
- **Index:** (data_delete_after) WHERE (status = 'canceled'::text)
- **Index:** (current_period_end) WHERE (status = ANY (ARRAY['active'::text, 'past_due'::text]))
- **Index:** (status, trial_ends_at, grace_ends_at)

subscription_events

History of every subscription change: trial, plan, seats, payment failure, read-only, reactivation, cancellation. Never changed.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Event ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
subscription_id	uuid	Yes	FK → subscriptions		Subscription
type	text	Yes			What happened
old_value	jsonb				Before, e.g. {"seats":10}
new_value	jsonb				After, e.g. {"seats":12}

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
actor_type	text	Yes			Who caused it
actor_user_id	uuid		FK → users		Tenant user (actor_type = user)
platform_admin_id	uuid		FK → platform_admins		Platform admin (actor_type = platform_admin)
note	text				Note, e.g. reason for a free extension
created_at	timestamptz	Yes		now()	Time

- **Unique:** (tenant_id, id)
- **Check:** ((type = ANY (['trial_started', 'activated', 'plan_changed', 'seats_changed', 'cycle_changed', 'payment_method_changed', 'payment_failed', 'grace_started', 'read_only', 'reactivated', 'cancel_requested', 'canceled', 'trial_extended', 'suspended', 'unsuspended', 'data_deleted'])))
- **Check:** (((actor_type = 'platform_admin') = (platform_admin_id IS NOT NULL)))
- **Check:** (((actor_type = 'user') = (actor_user_id IS NOT NULL)))
- **Check:** ((actor_type = ANY (['user', 'platform_admin', 'system', 'stripe'])))
- **Index:** (tenant_id, subscription_id, created_at)

billing_invoices

Our tax invoices to customers for their subscription (5% VAT), monthly or yearly.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Invoice ID
tenant_id	uuid	Yes	FK → tenants		Customer tenant
subscription_id	uuid	Yes	FK → subscriptions		Subscription
number	text	Yes			Tax invoice number, e.g. CRM-2026-000123
status	text	Yes		'open'	open = awaiting payment
period_start	date	Yes			Billed period start
period_end	date	Yes			Billed period end
issue_date	date	Yes		CURRENT_DATE	Issue date
due_date	date	Yes			Payment due date
currency	char(3)	Yes	FK → currencies	'AED'	Currency
subtotal	numeric(14,2)	Yes			Sum of lines excluding VAT
vat_rate	numeric(5,2)	Yes		5	VAT %
vat_amount	numeric(14,2)	Yes			VAT amount
total	numeric(14,2)			calculated: (subtotal + vat_amount)	Total including VAT
amount_paid	numeric(14,2)	Yes		0	Paid so far
balance	numeric(14,2)			calculated: ((subtotal + vat_amount) - amount_paid)	Amount still due
customer_legal_name	text	Yes			Customer name as on the invoice
customer_trn	text				Customer VAT number (TRN), if registered
customer_address	text				Customer address
pdf_file_id	uuid		FK → files		Generated PDF (files)

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
stripe_invoice_id	text				Stripe invoice
paid_at	timestamptz				When fully paid
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** (tenant_id, id)
- **Unique:** (stripe_invoice_id)
- **Unique:** (number)
- **Check:** ((vat_rate >= (0)))
- **Check:** ((status = ANY (['open', 'paid', 'void', 'uncollectible'])))
- **Check:** ((subtotal >= (0)))
- **Check:** (((status <> 'paid') OR ((amount_paid >= (subtotal + vat_amount)) AND (paid_at IS NOT NULL))))
- **Check:** ((due_date >= issue_date))
- **Check:** ((vat_amount >= (0)))
- **Check:** ((period_end >= period_start))
- **Check:** ((amount_paid >= (0)))
- **Index:** (due_date) WHERE (status = 'open'::text)
- **Index:** (tenant_id, issue_date DESC)

billing_invoice_lines

Lines of a subscription invoice: seats x price, extra seats added mid-period (pro-rata), credits.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Line ID
tenant_id	uuid	Yes	FK → tenants		Customer tenant
invoice_id	uuid	Yes	FK → billing_invoices		Invoice
line_no	smallint	Yes			Line order
type	text	Yes			Line type
description	text	Yes			e.g. Team plan, 10 users, Nov 2026
quantity	integer	Yes			Seats
unit_price	numeric(14,2)	Yes			Price per seat (negative for credits)
amount	numeric(14,2)	Yes			Line amount excluding VAT

- **Unique:** (tenant_id, invoice_id, line_no)
- **Check:** ((line_no > 0))
- **Check:** ((type = ANY (['subscription', 'extra_seats', 'credit'])))
- **Check:** ((quantity > 0))
- **Check:** (((type = 'credit') = (amount < (0))))
- **On delete:** invoice_id → billing_invoices: cascade

billing_payments

Card payments (from Stripe) and bank transfers (confirmed by the platform team) against subscription invoices.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Payment ID
tenant_id	uuid	Yes	FK → tenants		Customer tenant
invoice_id	uuid	Yes	FK → billing_invoices		Invoice paid
method	text	Yes			Payment method
status	text	Yes		'pending'	Payment status
amount	numeric(14,2)	Yes			Amount
currency	char(3)	Yes	FK → currencies	'AED'	Currency
stripe_payment_intent_id	text				Stripe payment (card)
bank_reference	text				Bank transfer reference
proof_file_id	uuid		FK → files		Transfer receipt uploaded by the customer (files)
confirmed_by	uuid		FK → platform_admins		Platform admin who confirmed a bank transfer
failure_reason	text				Why a card payment failed
received_at	timestamptz				When the money was received
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** (stripe_payment_intent_id)
- **Unique:** (tenant_id, id)
- **Check:** (((method <> 'bank_transfer') OR (status <> 'succeeded') OR (confirmed_by IS NOT NULL)))
- **Check:** (((status <> 'succeeded') OR (received_at IS NOT NULL)))
- **Check:** ((method = ANY (['card', 'bank_transfer'])))
- **Check:** ((status = ANY (['pending', 'succeeded', 'failed', 'refunded'])))
- **Check:** ((amount > (0)))
- **Check:** (((method <> 'card') OR (stripe_payment_intent_id IS NOT NULL)))
- **Index:** (tenant_id, invoice_id)
- **Index:** (created_at) WHERE ((status = 'pending'::text) AND (method = 'bank_transfer'::text))

tenant_usage_daily

Daily usage per tenant for the owner console: active users, leads, messages, storage.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
tenant_id	uuid	Yes	PK FK → tenants		Tenant
day	date	Yes	PK		Day (UAE time)
active_users	integer	Yes		0	Active users
leads_created	integer	Yes		0	New leads that day
messages_sent	integer	Yes		0	WhatsApp and email sent
storage_bytes	bigint	Yes		0	Files stored

- **Check:** ((active_users >= 0))
- **Check:** ((messages_sent >= 0))
- **Check:** ((storage_bytes >= 0))

- **Check:** `((leads_created >= 0))`

platform_admins

People who run the CRM business (owner console). Not part of any tenant.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Platform admin ID
auth_subject	text				Login provider user ID (separate Cognito pool, two-factor required)
email	citext	Yes			Login email
full_name	text	Yes			Full name
role	text	Yes			owner = everything; support = view tenants; billing = payments
status	text	Yes		'active'	Account status
last_login_at	timestamptz				Last successful login
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** `(email)`
- **Unique:** `(auth_subject)`
- **Check:** `((role = ANY (['owner', 'support', 'billing'])))`
- **Check:** `((status = ANY (['active', 'inactive'])))`

platform_settings

Business-wide settings: trial days, grace days, data retention, VAT rate.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
key	text	Yes	PK		Setting key, e.g. billing.grace_days
value	jsonb	Yes			Setting value
updated_by	uuid		FK → platform_admins		Last changed by
updated_at	timestamptz	Yes		now()	Last update time

- **Check:** `((key ~ '^[a-z_]+\.[a-z_]+$'))`

sales_enquiries

Enterprise and "talk to sales" requests from the website.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Enquiry ID
company_name	text	Yes			Company name
contact_name	text	Yes			Contact person
email	citext	Yes			Contact email
phone	text				Contact phone (E.164)
users_estimate	integer				Expected number of users
message	text				Message

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
status	text	Yes		'new'	Sales status
handled_by	uuid		FK → platform_admins		Platform admin handling it
tenant_id	uuid				Tenant created when won
created_at	timestamptz	Yes		now()	Received time
updated_at	timestamptz	Yes		now()	Last update time

- **Check:** (((users_estimate IS NULL) OR (users_estimate > 0)))
- **Check:** ((status = ANY (['new', 'contacted', 'won', 'lost'])))
- **Index:** (status, created_at DESC)

platform_audit_logs

Every owner-console action (view tenant, suspend, extend trial, confirm transfer, change plans). Never changed.

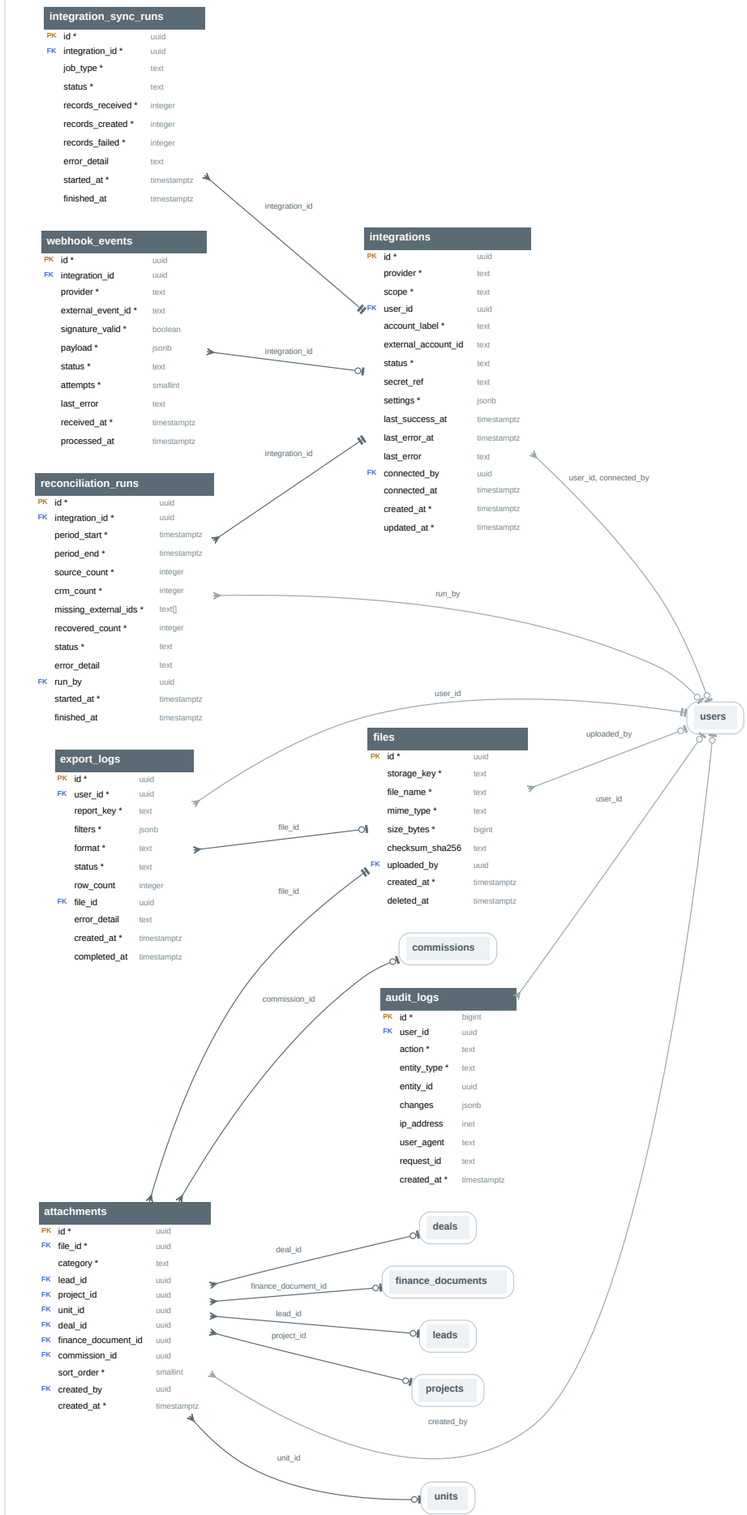
COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	bigint	Yes	PK		Audit ID (ordered)
platform_admin_id	uuid		FK → platform_admins		Acting platform admin (empty = system)
action	text	Yes			e.g. tenant.suspend, payment.confirm, plan.update
tenant_id	uuid		FK → tenants		Tenant affected (if any)
entity_type	text				Entity, e.g. subscriptions
entity_id	uuid				Row ID
changes	jsonb				Changed fields
ip_address	inet				Client IP
created_at	timestamptz	Yes		now()	Time

- **Index:** (tenant_id, created_at)

10 Integrations, files, reports and audit

In simple words: Connected accounts (Meta, portals, WhatsApp, Google, Microsoft, webhook), every incoming webhook, sync runs, Meta reconciliation, files, exports and the audit log.

Tables: integrations · webhook_events · integration_sync_runs · reconciliation_runs · files · attachments · export_logs · audit_logs



ER diagram: Integrations, files, reports and audit. Grey boxes are tables from other sections. tenant_id is hidden (every table belongs to a tenant).

integrations

Connected outside accounts (Meta, WhatsApp, portals, Google, Microsoft, general webhook). Tokens live in AWS SSM, not here.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Integration ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
provider	text	Yes			Provider (website/webhook = general webhook; google_workspace = Calendar + Meet; microsoft_365 = Calendar + Teams + mail)
scope	text	Yes		'tenant'	Company-wide or one user's account
user_id	uuid		FK → users		Owner user when scope = user
account_label	text	Yes			Display label, e.g. page or number
external_account_id	text				Account ID at the provider
status	text	Yes		'pending'	Connection status
secret_ref	text				AWS SSM parameter path holding the tokens
settings	jsonb	Yes		'{}'	Provider-specific settings
last_success_at	timestamptz				Last successful sync
last_error_at	timestamptz				Last error time
last_error	text				Last error detail
connected_by	uuid		FK → users		Connected by
connected_at	timestamptz				Connected time
created_at	timestamptz	Yes		now()	Created time
updated_at	timestamptz	Yes		now()	Last update time

- **Unique:** (tenant_id, id)
- **Unique:** Unique (empty values count as equal) (tenant_id, provider, external_account_id, user_id)
- **Check:** (((scope = 'user') = (user_id IS NOT NULL)))
- **Check:** ((status = ANY (['pending', 'connected', 'error', 'disconnected'])))
- **Check:** ((scope = ANY (['tenant', 'user'])))
- **Check:** ((provider = ANY (['meta', 'google_ads', 'linkedin', 'snapchat', 'tiktok', 'website', 'webhook', 'property_finder', 'bayut', 'dubizzle', 'whatsapp_cloud', 'email_ses', 'gmail', 'microsoft_365', 'google_workspace'])))

webhook_events

Every incoming webhook, stored before processing so nothing is lost and duplicates are ignored.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Event ID
tenant_id	uuid		FK → tenants		Tenant (empty until matched to an integration)
integration_id	uuid		FK → integrations		Matched integration
provider	text	Yes			Sending provider
external_event_id	text	Yes			Provider event ID (deduplication key)
signature_valid	boolean	Yes			Signature check result
payload	jsonb	Yes			Raw payload
status	text	Yes		'received'	Processing status

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
attempts	smallint	Yes		0	Processing attempts
last_error	text				Last processing error
received_at	timestamptz	Yes		now()	Received time
processed_at	timestamptz				Processed time

- **Unique:** (provider, external_event_id)
- **Check:** ((status = ANY (['received', 'processed', 'failed', 'ignored'])))
- **Index:** (status, received_at) WHERE (status = ANY (ARRAY['received'::text, 'failed'::text]))

integration_sync_runs

Each sync or import run with record counts and errors (shown on the Integrations screen).

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Run ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
integration_id	uuid	Yes	FK → integrations		Integration
job_type	text	Yes			e.g. lead_intake, spend_import, calendar_sync
status	text	Yes			Run result
records_received	integer	Yes		0	Records received
records_created	integer	Yes		0	Records created or updated
records_failed	integer	Yes		0	Records failed
error_detail	text				Error detail
started_at	timestamptz	Yes		now()	Start time
finished_at	timestamptz				End time

- **Unique:** (tenant_id, id)
- **Check:** ((status = ANY (['running', 'success', 'partial', 'failed'])))
- **Index:** (tenant_id, integration_id, started_at DESC)

reconciliation_runs

Meta lead reconciliation: leads at the source vs leads in the CRM, and missing IDs.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Run ID
tenant_id	uuid	Yes			Owning tenant
integration_id	uuid	Yes	FK → integrations		Ad account checked
period_start	timestamptz	Yes			Period from
period_end	timestamptz	Yes			Period to
source_count	integer	Yes			Leads at the source
crm_count	integer	Yes			Leads found in the CRM

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
missing_external_ids	text[]	Yes		'{}'	Source lead IDs not in the CRM
recovered_count	integer	Yes		0	Missing leads re-imported
status	text	Yes			Run status
error_detail	text				Sync error detail
run_by	uuid		FK → users		Started by (empty = nightly job)
started_at	timestamptz	Yes		now()	Start
finished_at	timestamptz				End

- **Check:** ((status = ANY (['running', 'completed', 'failed'])))
- **Check:** ((source_count >= 0))
- **Check:** ((crm_count >= 0))
- **Check:** ((recovered_count >= 0))
- **Check:** ((period_end > period_start))

files

Every uploaded or generated file. The file itself is stored in S3.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	File ID
tenant_id	uuid	Yes	FK → tenants		Owning tenant
storage_key	text	Yes			S3 object key
file_name	text	Yes			Original file name
mime_type	text	Yes			Content type
size_bytes	bigint	Yes			Size in bytes
checksum_sha256	text				Integrity checksum
uploaded_by	uuid		FK → users		Uploaded by (empty = system)
created_at	timestamptz	Yes		now()	Upload time
deleted_at	timestamptz				Soft delete time

- **Unique:** (storage_key)
- **Unique:** (tenant_id, id)
- **Check:** ((size_bytes >= 0))

attachments

Links a file to exactly one record: lead, project, unit, deal, financial document or commission.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Attachment ID
tenant_id	uuid	Yes			Owning tenant
file_id	uuid	Yes	FK → files		File
category	text	Yes			What the file is

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
lead_id	uuid		FK → leads		Attached to lead
project_id	uuid		FK → projects		Attached to project
unit_id	uuid		FK → units		Attached to unit
deal_id	uuid		FK → deals		Attached to deal
finance_document_id	uuid		FK → finance_documents		Attached to invoice / PO
commission_id	uuid		FK → commissions		Attached to commission
sort_order	smallint	Yes		0	Display order (image galleries)
created_by	uuid		FK → users		Attached by
created_at	timestamptz	Yes		now()	Attached time

- **Check:** `((num_nonnulls(lead_id, project_id, unit_id, deal_id, finance_document_id, commission_id) = 1))`
- **Check:** `((category = ANY (['document', 'image', 'brochure', 'floor_plan', 'contract', 'id_document', 'receipt', 'other'])))`
- **Index:** `(tenant_id, finance_document_id) WHERE (finance_document_id IS NOT NULL)`
- **Index:** `(tenant_id, lead_id) WHERE (lead_id IS NOT NULL)`
- **Index:** `(tenant_id, project_id) WHERE (project_id IS NOT NULL)`
- **Index:** `(tenant_id, unit_id) WHERE (unit_id IS NOT NULL)`

export_logs

Every export: who, which report, filters, format, rows, when.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	uuid	Yes	PK	auto (uuid)	Export ID
tenant_id	uuid	Yes			Owning tenant
user_id	uuid	Yes	FK → users		Exported by
report_key	text	Yes			Report, e.g. lead_master
filters	jsonb	Yes		'{}'	Filters applied
format	text	Yes			File format
status	text	Yes		'queued'	Status
row_count	integer				Rows exported
file_id	uuid		FK → files		Generated file
error_detail	text				Failure detail
created_at	timestamptz	Yes		now()	Requested time
completed_at	timestamptz				Completed time

- **Check:** `((format = ANY (['xlsx', 'csv'])))`
- **Check:** `((status = ANY (['queued', 'running', 'completed', 'failed'])))`
- **Check:** `((row_count >= 0))`
- **Check:** `(((status <> 'completed') OR ((file_id IS NOT NULL) AND (row_count IS NOT NULL))))`
- **Index:** `(tenant_id, created_at DESC)`

audit_logs

Who changed what and when, with before/after values. Never changed after saving.

COLUMN	TYPE	REQUIRED	KEY	DEFAULT	MEANING
id	bigint	Yes	PK		Audit ID (ordered)
tenant_id	uuid	Yes	FK → tenants		Owning tenant
user_id	uuid		FK → users		Acting user (empty = system)
action	text	Yes			Action
entity_type	text	Yes			Table / entity name, e.g. leads
entity_id	uuid				Row ID
changes	jsonb				Changed fields: {"field": [old, new]}
ip_address	inet				Client IP
user_agent	text				Client browser
request_id	text				Request trace ID
created_at	timestamptz	Yes		now()	Time

- **Check:** `((action = ANY (['create', 'update', 'delete', 'restore', 'merge', 'login', 'login_failed', 'logout', 'export', 'approve', 'cancel', 'permission_change', 'setting_change'])))`
- **Index:** `(tenant_id, entity_type, entity_id, created_at)`
- **Index:** `(tenant_id, created_at)`